

FRE7241 Algorithmic Portfolio Management

Lecture#2, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

November 4, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

Sell in May Calendar Strategy

Sell in May is a market timing calendar strategy, in which stocks are sold at the beginning of May, and then bought back at the beginning of November.

```
> # Calculate the positions
> retp <- na.omit(rutils::etfenv$returns$VTI)
> posv <- rep(NA_integer_, NROW(retp))
> datev <- zoo::index(retp)
> datev <- format(datev, "%m-%d")
> posv[datev == "05-01"] <- 0
> posv[datev == "05-03"] <- 0
> posv[datev == "11-01"] <- 1
> posv[datev == "11-03"] <- 1
> # Carry forward and backward non-NA posv
> posv <- zoo::na.locf(posv, na.rm=FALSE)
> posv <- zoo::na.locf(posv, fromLast=TRUE)
> # Calculate the strategy returns
> pnlmay <- posv*retp
> wealthv <- cbind(retp, pnlmay)
> colnames(wealthv) <- c("VTI", "sellmay")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```

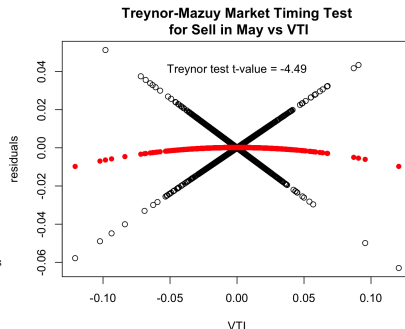


```
> # Plot wealth of Sell in May strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Sell in May Strategy")
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # OR: Open x11 for plotting
> x11(width=6, height=5)
> par(mar=c(4, 4, 3, 1), oma=c(0, 0, 0, 0))
> themev <- chart_theme()
> themev$col$line.col <- c("blue", "red")
> quantmod::chart_Series(wealthv, theme=themev, name="Sell in May Strategy")
> legend("topleft", legend=colnames(wealthv),
+   inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+   col=themev$col$line.col, bty="n")
```

Sell in May Strategy Market Timing

The *Sell in May* strategy doesn't demonstrate any ability of *timing* the VTI ETF.

```
> # Test if Sell in May strategy can time VTI
> desm <- cbind(wealthv, 0.5*(retp+abs(retp)), retp^2)
> colnames(desm) <- c(colnames(wealthv), "Merton", "Treyner")
> # Perform Merton-Henriksson test
> regmod <- lm(sellmay ~ VTI + Merton, data=desm)
> summary(regmod)
> # Perform Treynor-Mazuy test
> regmod <- lm(sellmay ~ VTI + Treynor, data=desm)
> summary(regmod)
> # Plot Treynor-Mazuy residual scatterplot
> resids <- (desm$sellmay - regmod$coeff["VTI"]*retp)
> plot.default(x=retp, y=resids, xlab="VTI", ylab="residuals")
> title(main="Treynor-Mazuy Market Timing Test\n for Sell in May vs")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retp
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retp, y=fitv, pch=16, col="red")
> text(x=0.0, y=0.8*max(resids), paste("Treynor test t-value =", tvalue))
```



Overnight Market Anomaly

The *Overnight Market Anomaly* is the consistent outperformance of overnight returns relative to the daytime returns.

The *Overnight Market Anomaly* has been observed for many decades for most stock market indices, but not always for all stock sectors.

The Overnight Strategy consists of holding a long position only overnight (buying at the close and selling at the open the next day).

The Daytime Strategy consists of holding a long position only during the daytime (buying at the open and selling at the close the same day).

The *Overnight Market Anomaly* is not as pronounced after the 2008–2009 financial crisis.

```
> # Calculate the log of OHLC VTI prices
> ohlc <- log(rutils::etfenv$VTI)
> openp <- quantmod::Op(ohlc)
> highp <- quantmod::Hi(ohlc)
> lowp <- quantmod::Lo(ohlc)
> closep <- quantmod::Cl(ohlc)
> # Calculate the close-to-close log returns,
> # the daytime open-to-close returns
> # and the overnight close-to-open returns.
> retp <- rutils::diffit(closep)
> colnames(retp) <- "daily"
> retc <- (closep - openp)
> colnames(retc) <- "daytime"
> reton <- (openp - rutils::lagit(closep, lagg=1, pad_zeros=FALSE))
> colnames(reton) <- "overnight"
```

Wealth of Close-to-Close, Overnight, and Daytime Strategies



```
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retp, reton, retc)
> sqrt(252)*sapply(wealthv, function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot the log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+ main="Wealth of Close-to-Close, Overnight, and Daytime Strategies")
+ dySeries(name="daily", strokeWidth=2, col="blue") %>%
+ dySeries(name="overnight", strokeWidth=2, col="red") %>%
+ dySeries(name="daytime", strokeWidth=2, col="green") %>%
+ dyLegend(width=500)
```

Turn of the Month Effect

The *Turn of the Month* (TOM) effect is the outperformance of stocks on the last trading day of the month and on the first three days of the following month.

The TOM effect was observed for the period from 1928 to 1975, but it has been less pronounced since the year 2000.

The TOM effect has been attributed to the investment of funds deposited at the end of the month.

This would explain why the TOM effect has been more pronounced for less liquid small-cap stocks.

```
> # Calculate the VTI returns
> retp <- na.omit(rutils::returns$VTI)
> datev <- zoo::index(retp)
> # Calculate the first business day of every month
> dayv <- as.numeric(format(datev, "%d"))
> indeks <- which(rutils::diffit(dayv) < 0)
> datev[head(indeks)]
> # Calculate the Turn of the Month dates
> indeks <- lapply((-1):2, function(x) indeks + x)
> indeks <- do.call(c, indeks)
> sum(indeks > NROW(datev))
> indeks <- sort(indeks)
> datev[head(indeks, 11)]
> # Calculate the Turn of the Month pnls
> pnls <- numeric(NROW(retp))
> pnls[indeks] <- retp[indeks, ]
```



```
> # Combine data
> wealthv <- cbind(retp, pnls)
> colv <- c("VTI", "TOM Strategy")
> colnames(wealthv) <- colv
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # dygraph plot VTI Turn of the Month strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Turn of the Month Strategy") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red")
```

The Stop-loss Strategy

Stop-loss rules are used to reduce losses in case of a significant drawdown in prices.

For example, a simple stop-loss rule is to sell the stock if its price drops below the stop-loss level, equal to the stop-loss percentage times the previous maximum price.

The stock is bought back after the price recovers, for example after the price reaches its previous maximum price.

The stop-loss strategy trades a single \$1 of stock, and is either long \$1 of stock or it's flat (\$0 of stock).

The stop-loss strategy is trend-following because it expects prices to continue dropping.

```
> # Calculate the log VTI prices and percentage returns
> pricev <- log(na.omit(rutils::etfenv$prices$VTI))
> nrow <- NROW(pricev)
> datev <- zoo::index(pricev)
> retv <- rutils::diffit(pricev)
> # Simulate stop-loss strategy
> stopl <- 0.05 # Stop-loss percentage
> pricem <- cummax(pricev) # Trailing maximum prices
> # Calculate the drawdown
> dd <- (pricev - pricem)
> pnls <- retv # Initialize PnLs
> for (i in 1:(nrow-1)) {
+ # Check for stop-loss
+   if (dd[i] < -stopl*pricem[i])
+     pnls[i+1] <- 0 # Set PnLs = 0 if in stop-loss
+ } # end for
> # Same but without using loops in R
> pnls2 <- retv
> insl <- rutils::lagit(dd < -stopl*pricem)
> pnls2 <- ifelse(insl, 0, pnls2)
> all.equal(pnls, pnls2, check.attributes=FALSE)
```

Stop-loss Strategy Performance

The stop-loss strategy underperforms because it trades with a lag.

After it hits the stop-loss, it unwinds its long position with a lag. And after it recovers from the stop-loss, it re-enters the long position with a lag.

The losses are compounded when the strategy is "whipsawed", when prices are range-bound without having a trend, and the strategy switches back and forth.

When prices are range-bound without a trend, the stop-loss strategy often stops because of a drawdown (goes flat risk), but if the prices soon rebound, then it's forced to buy back the stock.

The strategy enters the stop-loss just before the prices start rising, and it re-enters the long position just before the prices start dropping.

```
> # Combine the data
> wealthv <- cbind(retsp, pnlsl)
> colv <- c("VTI", "Strategy")
> colnames(wealthv) <- colv
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # dygraph plot the stop-loss strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="VTI Stop-loss Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

VTI Stop-loss Strategy



```
> # Plot the stop-loss strategy with shading
> indic <- (rutils::diffit(insl) != 0) # Indices of stop-loss
> crossd <- c(datev[indic], datev[nrows]) # Dates of stop-loss
> shadev <- ifelse(indic[indic] == 1, "antiquewhite", "lightgreen")
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main="VTI Stop-loss Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

Optimal Stop-loss Strategy

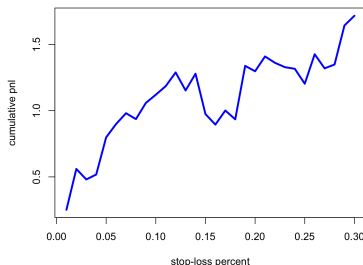
Stop-loss rules can reduce the largest drawdowns but they also tend to reduce cumulative returns for stocks with good returns.

The best performing stop-loss strategy for *VTI* has the largest stop-loss percentage - i.e. it almost never enters into a stop-loss.

That's because *VTI* has had positive returns in the last 20 years, so a stop-loss rule had little benefit.

That's why many quantitative investment funds do not use stop-loss rules if they have strong convictions that the stocks will have positive returns.

Cumulative PnLs for Stop-loss Strategies



```
> # Simulate multiple stop-loss strategies
> dd <- (pricev - pricem)
> stopv <- 0.01*(1:30)
> pnlc <- sapply(stopv, function(stopl) {
+   pnls <- retp
+   insl <- rutils::lagit(dd < -stopl*pricem)
+   pnls <- ifelse(insl, 0, pnls)
+   sum(pnls)
+ }) # end sapply
```

```
> # Plot cumulative PnLs of stop-loss strategies
> plot(x=stopv, y=pnlc,
+   main="Cumulative PnLs of Stop-loss Strategies",
+   xlab="stop-loss percent", ylab="cumulative pnl",
+   t="l", lwd=3, col="blue")
```


Stop-Start Strategy

The stop-loss strategy can be improved by introducing a start-gain rule: buy back the stock if its price rebounds from the previous minimum and exceeds the start-gain level.

The stop-start strategy implements both stop-loss events due to price drawdowns and start-gain events due to price draw-ups.

In order to determine the stop-loss and start-gain events, the stop-start strategy follows the trailing maximum and trailing minimum prices.

A start-gain event is when the stock price rebounds from the previous minimum and exceeds the start-gain level, equal to the start-gain percentage times the previous minimum price.

After the start-gain level is crossed, the strategy buys back the stock which was sold under the stop-loss.

The stop-start strategy is trend-following because it profits if price trends are persistent. But it loses if prices are range-bound.

```
> # Function for simulating the stop-start strategy
> stopstart <- function(stopl) {
+   maxp <- pricev[1] # Trailing maximum price
+   minp <- pricev[1] # Trailing minimum price
+   insl <- FALSE # Is in stop-loss?
+   insg <- FALSE # Is in start-gain?
+   pnls <- retp # Initialize PnLs
+   for (i in 1:nrows) {
+     if (insl) { # In stop-loss
+       pnls[i] <- 0 # Set PnLs = 0 if in stop-loss
+       minp <- min(minp, pricev[i]) # Update minimum price to current price
+       if (pricev[i] > (1 + stopl)*minp) { # Check for start-gain
+         insg <- TRUE # Is in start-gain?
+         insl <- FALSE # Is in stop-loss?
+         maxp <- pricev[i] # Reset trailing maximum price
+       } # end if
+     } else if (insg) { # In start-gain
+       maxp <- max(maxp, pricev[i]) # Update maximum price to current price
+       if (pricev[i] < (1 - stopl)*maxp) { # Check for stop-loss
+         insl <- TRUE # Is in stop-loss?
+         insg <- FALSE # Is in start-gain?
+         minp <- pricev[i] # Reset trailing minimum price
+       } # end if
+     } else { # Warmup period
+       # Update the maximum and minimum prices
+       maxp <- max(maxp, pricev[i])
+       minp <- min(minp, pricev[i])
+       # Update the stop-loss and start-gain indicators
+       insl <- (pricev[i] < (1 - stopl)*maxp) # Is in stop-loss?
+       insg <- (pricev[i] > (1 + stopl)*minp) # Is in start-gain?
+     } # end if
+   } # end for
+   return(pnls)
+ } # end stopstart
```

Stop-Start Strategy Performance

The stop-start strategy spends less time in a stop-loss because prices tend to rebound sharply after a steep loss.

The start-gain rule tends to quickly override the stop-loss rule, so that the strategy is long the stock after it recovers after a stop-loss.

```
> # Simulate the stop-start strategy
> pnls <- stopstart(0.07)
> # Combine the data
> wealthv <- cbind(retp, pnls)
> colv <- c("VTI", "Strategy")
> colnames(wealthv) <- colv
> # Has low correlation to VTI
> cor(wealthv)[1, 2]
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # dygraph plot the stop-loss strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="VTI Stop-Start Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

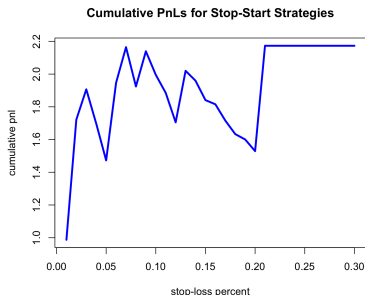


```
> # Plot the stop-start strategy with shading
> insl <- (pnls == 0) # Is in stop-loss?
> indic <- (rutils::diffit(insl) != 0) # Indices of crosses
> crossd <- c(datev[indic], datev[nrows]) # Dates of crosses
> shadev <- ifelse(insl[indic] == 1, "antiquewhite", "lightgreen")
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main="VTI Stop-Start Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

Optimal Stop-Start Strategy

The stop-start strategy performs much better than the stop-loss strategy because it captures stock gains.

```
> # Simulate multiple stop-loss strategies in a loop (slow)
> stopv <- 0.01*(1:30)
> # pnlc <- sapply(stopv, function(stopl) {
> #   sum(stopstart(stopl))
> # }) # end sapply
> # Simulate stop-loss strategies in a parallel loop (faster)
> library(parallel)
> ncores <- detectCores()-1
> pnlc <- unlist(mclapply(stopv, function(stopl) {
+   sum(stopstart(stopl))
+ }, mc.cores=ncores)) # end mclapply
> # Find optimal stop-loss level
> stopl <- stopv[which.max(pnlc)]
```

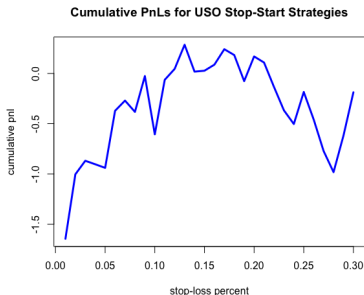


```
> # Plot cumulative PnLs of stop-loss strategies
> plot(x=stopv, y=pnlc,
+   main="Cumulative PnLs of Stop-Start Strategies",
+   xlab="stop-loss percent", ylab="cumulative pnl",
+   t="l", lwd=3, col="blue")
```

Stop-Start Strategy for Other ETFs

The stop-start strategy can prevent losses for stocks with significant negative returns, like the *USO* ETF (oil fund).

```
> # Calculate the USO prices and returns
> pricev <- log(na.omit(rutils::etfenv$prices$USO))
> nrow <- NROW(pricev)
> datev <- zoo::index(pricev)
> retp <- rutils::diffit(pricev)
> # Simulate multiple stop-start strategies
> stopv <- 0.01*(1:30)
> pnlc <- unlist(mclapply(stopv, function(stopl) {
+   sum(stopstart(stopl))
+ }, mc.cores=ncores)) # end mclapply
```



```
> # Plot cumulative PnLs of stop-start strategies
> plot(x=stopv, y=pnlc,
+   main="Cumulative PnLs of USO Stop-Start Strategies",
+   xlab="stop-loss percent", ylab="cumulative pnl",
+   t="l", lwd=3, col="blue")
```

Optimal Stop-Start Strategy For *USO*

The stop-start strategy for the *USO* ETF has performed well because *USO* has had very negative returns in the last 20 years.

Stop-start strategies are able to preserve profits for the best performing stocks, and avoid losses for the worst performing stocks.

```
> # Simulate optimal stop-start strategy for USO
> stop1 <- stopv[which.max(pnlc)]
> pnls <- stopstart(stop1)
> # Combine the data
> wealthv <- cbind(retp, pnls)
> colv <- c("USO", "Strategy")
> colnames(wealthv) <- colv
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # dygraph plot the stop-start strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="USO Stop-Start Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



```
> # Plot the stop-start strategy with shading
> insl <- (pnls == 0) # Is in stop-loss?
> indic <- (rutils::diffit(insl) != 0) # Indices of crosses
> crossd <- c(datev[indic], datev[nrows]) # Dates of crosses
> shadef <- ifelse(insl[indic] == 1, "antiquewhite", "lightgreen")
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main="USO Stop-Start Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadef)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

Trend-Flip Strategy

The trend-flip strategy shorts the stock in a drawdown, instead of going flat like the stop-start strategy.

In order to determine the stop-loss and start-gain events, the trend-flip strategy follows the trailing maximum and trailing minimum prices.

A start-gain event is when the stock price rebounds from the previous minimum and exceeds the start-gain level, equal to the start-gain percentage times the previous minimum price.

After the start-gain level is crossed, the strategy buys back the stock which was sold under the stop-loss.

The trend-flip strategy is trend-following because it profits if price trends are persistent. But it loses if prices are range-bound.

```
> # Function for simulating the trend-flip strategy
> trend_flip <- function(stopl = 0.1) {
+   pricev <- coredata(pricev)
+   nrows <- NROW(pricev)
+   pricec <- pricev[1]
+   pricemax <- pricec
+   pricemin <- pricec
+   drawp <- 0
+   gainp <- 0
+   posv <- integer(nrows)
+   pnlv <- numeric(nrows)
+   for (it in 2:(nrows - 1)) {
+     pricec <- pricev[it]
+     pnlv[it] <- posv[it] * (pricec - pricev[it - 1])
+     pricemax <- max(pricemax, pricec)
+     pricemin <- min(pricemin, pricec)
+     drawp <- min(pricec - pricemax, 0)
+     gainp <- max(pricec - pricemin, 0)
+     if ((drawp < -stopl) & (posv[it] >= 0)) {
+       posv[it + 1] <- (-1)
+       pricemin <- pricec
+     } else if ((gainp > stopl) & (posv[it] <= 0)) {
+       posv[it + 1] <- 1
+       pricemax <- pricec
+     } else { # Do nothing
+       posv[it + 1] <- posv[it]
+     }
+   }
+   return(cbind(pnlv, posv))
+ } # end trend_flip
```

Trend-Flip Strategy Performance

The trend-flip strategy spends less time in a stop-loss because prices tend to rebound sharply after a steep loss.

The start-gain rule tends to quickly override the stop-loss rule, so that the strategy is long the stock after it recovers after a stop-loss.

```
> # Simulate the trend-flip strategy
> pnls <- trend_flip(0.09)
> # Combine the data
> wealthv <- cbind(retp, pnls[, "pnlv"])
> colv <- c("VTI", "Strategy")
> colnames(wealthv) <- colv
> # Has low correlation to VTI
> cor(wealthv)[1, 2]
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # dygraph plot the stop-loss strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="VTI Trend-Flip Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

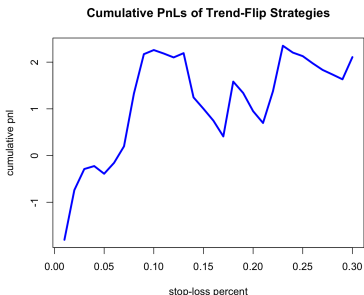


```
> # Plot the trend-flip strategy with shading
> posv <- pnls[, "posv"]
> indic <- (rutils::diffit(posv) != 0) # Indices of crosses
> crossd <- c(datev[indic], datev[nrows]) # Dates of crosses
> shadev <- ifelse(posv[indic] == 1, "lightgreen", "antiquewhite")
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main="VTI Trend-Flip Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

Optimal Trend-Flip Strategy

The trend-flip strategy has a low correlation with the stock returns because it can be either long or short the stock.

```
> # Simulate stop-loss strategies in a parallel loop
> stopv <- 0.01*(1:30)
> library(parallel)
> ncores <- detectCores()-1
> pnlc <- unlist(mclapply(stopv, function(stopl) {
+   sum(trend_flip(stopl)[, "pnlv"])
+ }, mc.cores=ncores)) # end mclapply
> # Find optimal stop-loss level
> stopl <- stopv[which.max(pnlc)]
```

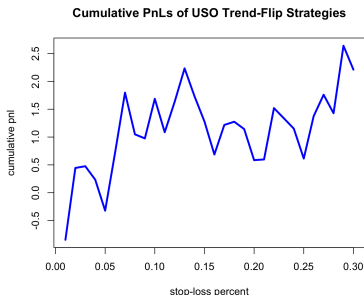


```
> # Plot cumulative PnLs of trend-flip strategies
> plot(x=stopv, y=pnlc,
+   main="Cumulative PnLs of Trend-Flip Strategies",
+   xlab="stop-loss percent", ylab="cumulative pnl",
+   t="l", lwd=3, col="blue")
```


Trend-Flip Strategy for Other ETFs

The trend-flip strategy can prevent losses for stocks with significant negative returns, like the *USO* ETF (oil fund).

```
> # Calculate the USO prices and returns
> pricev <- log(na.omit(rutils::etfenv$prices$USO))
> nrow <- NROW(pricev)
> datev <- zoo::index(pricev)
> retv <- rutils::diffit(pricev)
> # Simulate multiple trend-flip strategies
> stopv <- 0.01*(1:30)
> pnlc <- unlist(mclapply(stopv, function(stopl) {
+   sum(trend_flip(stopl)[, "pnlv"])
+ }, mc.cores=ncores)) # end mclapply
```



```
> # Plot cumulative PnLs of trend-flip strategies
> plot(x=stopv, y=pnlc,
+   main="Cumulative PnLs of USO Trend-Flip Strategies",
+   xlab="stop-loss percent", ylab="cumulative pnl",
+   t="l", lwd=3, col="blue")
```

Optimal Trend-Flip Strategy For *USO*

The trend-flip strategy for the *USO* ETF has performed well because *USO* has had very negative returns in the last 20 years.

Trend-flip strategies are able to preserve profits for the best performing stocks, and avoid losses for the worst performing stocks.

```
> # Simulate optimal trend-flip strategy for USO
> stopl <- stopv[which.max(pnlc)]
> pnls <- trend_flip(stopl)
> # Combine the data
> wealthv <- cbind(retp, pnls[, "pnlv"])
> colv <- c("USO", "Strategy")
> colnames(wealthv) <- colv
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # dygraph plot the trend-flip strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="USO Trend-Flip Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



```
> # Plot the trend-flip strategy with shading
> posv <- pnls[, "posv"]
> indic <- (rutils::diffit(posv) != 0) # Indices of crosses
> crossd <- c(datev[indic], datev[nrows]) # Dates of crosses
> shadev <- ifelse(posv[indic] == 1, "lightgreen", "antiquewhite")
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main="USO Trend-Flip Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

EMA Price Technical Indicator

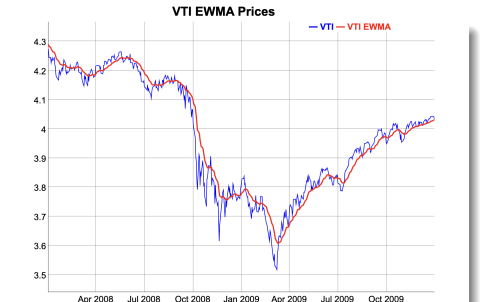
The *Exponentially Weighted Moving Average Price* (*EMA*) is defined as the weighted average of prices over a rolling interval:

$$p_i^{EMA} = (1 - \lambda) \sum_{j=0}^{\infty} \lambda^j p_{i-j}$$

Where the decay factor λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent prices, and vice versa.

The function `HighFreq::roll_wsum()` calculates the convolution of a time series with a vector of weights.

```
> # Extract the log VTI prices
> ohlc <- log(rutils::etfenv$VTI)
> closep <- quantmod::Cl(ohlc)
> colnames(closep) <- "VTI"
> nrows <- NROW(closep)
> # Calculate the EMA weights
> lookb <- 111
> lambdaf <- 0.9
> weightv <- lambdaf^(0:lookb)
> weightv <- weightv/sum(weightv)
> # Calculate the EMA prices as a convolution
> pricema <- HighFreq::roll_wsum(closep, weightv=weightv)
> pricev <- cbind(closep, pricema)
> colnames(pricev) <- c("VTI", "VTI EMA")
```



```
> # Dygraphs plot with custom line colors
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="VTI EMA Prices") %>%
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
> # Standard plot of EMA prices with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> colorv <- c("blue", "red")
> themev$col$line.col <- colorv
> quantmod::chart_Series(pricev["2009"], theme=themev,
+   lwd=2, name="VTI EMA Prices")
> legend("topleft", legend=colnames(pricev), y.intersp=0.5,
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

Recursive EMA Price Indicator

The *EMA* prices can be calculated recursively as follows:

$$p_i^{EMA} = \lambda p_{i-1}^{EMA} + (1 - \lambda)p_i$$

Where the decay factor λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent prices, and vice versa.

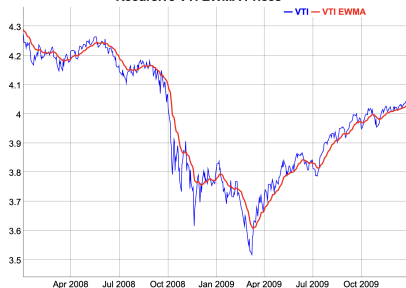
The recursive *EMA* prices are slightly different from those calculated as a convolution, because the convolution uses a fixed look-back interval.

The compiled C++ function `stats::C_rfilter()` calculates the exponentially weighted moving average prices recursively.

The function `HighFreq::run_mean()` calculates the exponentially weighted moving average prices recursively.

```
> # Calculate the EMA prices recursively using C++ code
> emar <- .Call(stats::C_rfilter, closep, lambdaf, c(as.numeric(c
> # Or R code
> emar <- filter(closep, filter=lambdaf, init=as.numeric(closep[
> emar <- (1-lambdaf)*emar
> # Calculate the EMA prices recursively using RcppArmadillo
> pricema <- HighFreq::run_mean(closep, lambda=lambdaf)
> all.equal(drop(pricema), emar)
> # Compare the speed of C++ code with RcppArmadillo
> library(microbenchmark)
> summary(microbenchmark(
+   Rcnp=HighFreq::run_mean(closep, lambda=lambdaf),
+   rfilter=.Call(stats::C_rfilter, closep, lambdaf, c(as.numeric(c
```

Recursive VTI EWMA Prices



```
> # Dygraphs plot with custom line colors
> pricev <- cbind(closep, pricema)
> colnames(pricev) <- c("VTI", "VTI EMA")
> colv <- colnames(pricev)
> dygraphs::dygraph(pricev["2008/2009"], main="Recursive VTI EMA Prices")
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
> # Standard plot of EMA prices with custom line colors
> themev <- chart_theme()
> colorv <- c("blue", "red")
> themev$col$line.col <- colorv
> quantmod::chart_Series(pricev["2009"], theme=themev,
+   lwd=2, name="VTI EMA Prices")
> legend("topleft", legend=colnames(pricev), y.intersp=0.5,
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

The EMA Crossover Strategy

The trend following *EMA Crossover* strategy switches its stock position depending if the current price is above or below the *EMA*.

If the stock price is above the *EMA* price, then the strategy switches to long \$1 dollar of stock, and if it is below, to short \$1 dollar of stock.

The strategy holds the same position until the *EMA* crosses over the current price (either from above or below), and then it switches its position.

The strategy is therefore always either long \$1 dollar of stock or short \$1 dollar of stock.

```
> # Calculate the EMA prices recursively using C++ code
> lambdaf <- 0.984
> pricema <- HighFreq::run_mean(closep, lambda=lambdaf)
> pricev <- cbind(closep, pricema)
> colnames(pricev) <- c("VTI", "VTI EMA")
> colv <- colnames(pricev)
> # Calculate the positions, either: -1, 0, or 1
> indic <- sign(closep - pricema)
> posv <- rutils::lagit(indic, lagg=1)
> # Create colors for background shading
> crossd <- (rutils::diffit(posv) != 0)
> shadev <- posv[crossd]
> crossd <- c(zoo::index(shadev), end(posv))
> shadev <- ifelse(drop(zoo::coredata(shadev)) == 1, "lightgreen",
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(pricev, main="VTI EMA Prices") %>%
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=3, col="red") %>%
+   dyLegend(show="always", width=300)
```



```
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1],
+ } # end for
> # Plot the dygraph object
> dyplot
> # Standard plot of EMA prices with position shading
> quantmod::chart_Series(pricev, theme=themev,
+   lwd=2, name="VTI EMA Prices")
> add_TA(posv > 0, on=-1, col="lightgreen", border="lightgreen")
> add_TA(posv < 0, on=-1, col="lightgrey", border="lightgrey")
> legend("topleft", legend=colnames(pricev),
+   inset=0.1, bg="white", lty=1, lwd=6, y.intersp=0.5,
+   col=themev$col$line.col, bty="n")
```

EMA Crossover Strategy Performance

The crossover strategy trades at the *Close* price on the same day that prices cross the *EMA*, which may be difficult in practice.

The crossover strategy performance is worse than the underlying asset (*VTI*), but it has a negative correlation to it, which is very valuable when building a portfolio.

```
> # Calculate the daily profits and losses of crossover strategy
> retp <- rutils::diffit(closep) # VTI returns
> pnls <- retp*posv
> colnames(pnls) <- "EMA"
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "EMA PnL")
> # Annualized Sharpe ratio of crossover strategy
> sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
> # The crossover strategy has a negative correlation to VTI
> cor(wealthv)[1, 2]
> # Plot dygraph of crossover strategy wealth
> # Create dygraph object without plotting it
> colorv <- c("blue", "red")
> dyplot <- dygraphs::dygraph(cumsum(wealthv), main="Performance of Crossover Strategy")
+ dyOptions(colors=colorv, strokeWidth=2) %>%
+ dyLegend(show="always", width=300)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>%
+     dyShading(from=crossd[i], to=crossd[i+1], color=shadev[i])
+ } # end for
> # Plot the dygraph object
> dyplot
```

Performance of EWMA Strategy



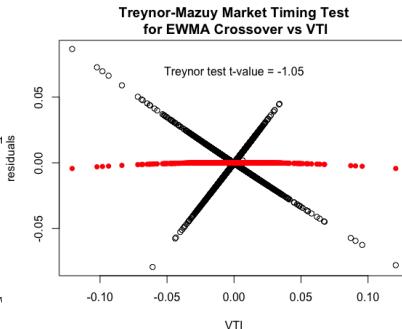
```
> # Standard plot of crossover strategy wealth
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(cumsum(wealthv), theme=themev,
+   name="Performance of Crossover Strategy")
> add_TA(posv > 0, on=-1, col="lightgreen", border="lightgreen")
> add_TA(posv < 0, on=-1, col="lightgrey", border="lightgrey")
> legend("top", legend=colnames(wealthv), y.intersp=0.5,
+   inset=0.05, bg="white", lty=1, lwd=6,
+   col=themev$col$line.col, bty="n")
```

EMA Crossover Strategy Market Timing Skill

The EMA crossover strategy shorts the market during significant selloffs, but otherwise doesn't display market timing skill.

The t-value of the *Treynor-Mazuy* test is negative, but not statistically significant.

```
> # Test EMA crossover market timing of VTI using Treynor-Mazuy test
> desm <- cbind(pnlts, retp, retp^2)
> desm <- na.omit(desm)
> colnames(desm) <- c("EMA", "VTI", "Treynor")
> regmod <- lm(EMA ~ VTI + Treynor, data=desm)
> summary(regmod)
> # Plot residual scatterplot
> resid <- (desm$EMA - regmod$coeff["VTI"]*retp)
> resid <- regmod$residuals
> plot.default(x=retp, y=resid, xlab="VTI", ylab="residuals")
> title(main="Treynor-Mazuy Market Timing Test\n for EMA Crossover")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retp
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retp, y=fitv, pch=16, col="red")
> text(x=0.0, y=0.8*max(resid), paste("Treynor test t-value =", tvalue))
```



EMA Crossover Strategy With Lag

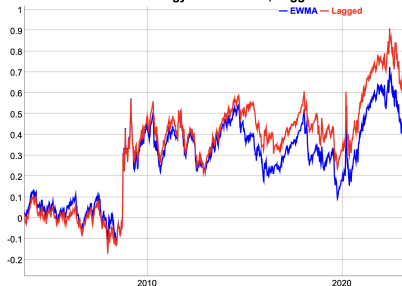
The crossover strategy suffers losses when prices are range-bound without a trend, because whenever it switches position the prices soon change direction. (This is called a "whipsaw".)

To prevent whipsaws and over-trading, the crossover strategy may choose to delay switching positions until the indicator repeats the same value for several periods.

There's a tradeoff between switching positions too early and risking a whipsaw, and waiting too long and missing an emerging trend.

```
> # Determine the trade dates right after EMA has crossed prices
> indic <- sign(closep - pricema)
> # Calculate the positions from lagged indicator
> lagg <- 2
> indic <- HighFreq::roll_sum(indic, lagg)
> # Calculate the positions, either: -1, 0, or 1
> posv <- rep(NA_integer_, nrows)
> posv[1] <- 0
> posv <- ifelse(indic == lagg, 1, posv)
> posv <- ifelse(indic == (-lagg), -1, posv)
> posv <- zoo::na.locf(posv, na.rm=FALSE)
> posv <- xts::xts(posv, order.by=zoo::index(closep))
> # Lag the positions to trade in next period
> posv <- rutils::lagit(posv, lagg=1)
> # Calculate the PnLs of lagged strategy
> pnlslag <- retp*posv
> colnames(pnlslag) <- "Lagged Strategy"
```

EWMA Crossover Strategy EWMA=0.104, Lagged=0.151



```
> wealthv <- cbind(pnlsl, pnlslag)
> colnames(wealthv) <- c("EMA", "Lagged")
> # Annualized Sharpe ratios of crossover strategies
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
> # Plot both strategies
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main=paste("EMA Crossover",
+ dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+ dyLegend(show="always", width=300))
```


Crossover Strategy Trading at the Open Price

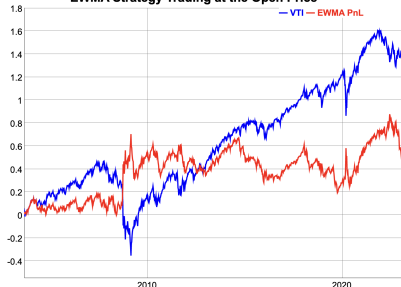
In practice it may not be possible to trade immediately at the *Close* price on the same day that prices cross the *EMA*.

Then the strategy may trade at the *Open* price on the next day.

The Profit and Loss (*PnL*) on a trade date is the sum of the realized *PnL* from closing the old position, plus the unrealized *PnL* after opening the new position.

```
> # Calculate the positions, either: -1, 0, or 1
> indic <- sign(closep - pricema)
> posv <- rutils::lagit(indic, lagg=1)
> # Calculate the daily pnl for days without trades
> pnls <- retp*posv
> # Determine the trade dates right after EMA has crossed prices
> crossd <- which(rutils::diffit(posv) != 0)
> # Calculate the realized pnl for days with trades
> openp <- quantmod::Op(ohlc)
> closelag <- rutils::lagit(closep)
> poslag <- rutils::lagit(posv)
> pnls[crossd] <- poslag[crossd]*(openp[crossd] - closelag[crossd])
> # Calculate the unrealized pnl for days with trades
> pnls[crossd] <- pnls[crossd] +
+   posv[crossd]*(closep[crossd] - openp[crossd])
> # Calculate the wealth
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "EMA PnL")
> # Annualized Sharpe ratio of crossover strategy
> sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
> # The crossover strategy has a negative correlation to VTI
> cor(wealthv)[1, 2]
```

EWMA Strategy Trading at the Open Price



```
> # Plot dygraph of crossover strategy wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Crossover Strategy
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Standard plot of crossover strategy wealth
> quantmod::chart_Series(cumsum(wealthv)[endw], theme=themev,
+   name="Crossover Strategy Trading at the Open Price")
> legend("top", legend=colnames(wealthv),
+   inset=0.05, bg="white", lty=1, lwd=6,
+   col=themev$col$line.col, bty="n")
```

EMA Crossover Strategy With Transaction Costs

The *bid-ask spread* is the percentage difference between the *ask* (offer) minus the *bid* prices, divided by the *mid* price.

The bid-ask spread for many liquid ETFs is about 1 basis point. For example the *XLK ETF*

Let n_t be the number of shares of the stock owned at time t , and let p_t be their price.

Then the traded dollar amount of the stock is equal to the change in the number of shares times the stock price: $\Delta n_t p_t$.

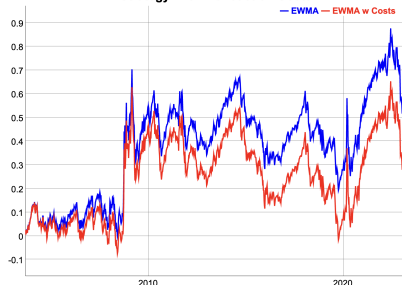
The *transaction costs* c^r due to the *bid-ask spread* are equal to half the *bid-ask spread* δ times the absolute value of the traded dollar amount of the stock:

$$c^r = \frac{\delta}{2} |\Delta n_t| p_t$$

If d_t is the dollar amount of the stock owned at time t then the *transaction costs* c^r are equal to:

$$c^r = \frac{\delta}{2} |\Delta d_t|$$

EWMA Strategy With Transaction Costs



```
> # The bid-ask spread is equal to 1 bp for liquid ETFs
> bidask <- 0.001
> # Calculate the transaction costs
> costv <- 0.5*bidask*abs(poslag - posv)
> # Plot strategy with transaction costs
> wealthv <- cbind(pnlsl, pnlsl - costv)
> colnames(wealthv) <- c("EMA", "EMA w Costs")
> colorv <- c("blue", "red")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Crossover Strategy
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Simulation Function for EMA Crossover Strategy

The *EMA* strategy can be simulated by a single function, which allows the analysis of its performance depending on its parameters.

The function `sim_ema()` performs a simulation of the *EMA* strategy, given an *OHLC* time series of prices, and a decay factor λ .

The function `sim_ema()` returns the *EMA* strategy positions and returns, in a two-column *xts* time series.

```
> sim_ema <- function(closep, lambdaf=0.9, bidask=0.001, trend=1, l
+   retp <- rutils::diffit(closep)
+   nrows <- NROW(closep)
+   # Calculate the EMA prices
+   pricema <- HighFreq::run_mean(closep, lambda=lambdaf)
+   # Calculate the indicator
+   indic <- trend*sign(closep - pricema)
+   if (lagg > 1) {
+     indic <- HighFreq::roll_sum(indic, lagg)
+     indic[1:lagg] <- 0
+   } # end if
+   # Calculate the positions, either: -1, 0, or 1
+   posv <- rep(NA_integer_, nrows)
+   posv[1] <- 0
+   posv <- ifelse(indic == lagg, 1, posv)
+   posv <- ifelse(indic == (-lagg), -1, posv)
+   posv <- zoo::na.locf(posv, na.rm=FALSE)
+   posv <- xts::xts(posv, order.by=zoo::index(closep))
+   # Lag the positions to trade on next day
+   posv <- rutils::lagit(posv, lagg=1)
+   # Calculate the PnLs of strategy
+   pnls <- retp*posv
+   costv <- 0.5*bidask*abs(rutils::diffit(posv))
+   pnls <- (pnls - costv)
+   # Calculate the strategy returns
+   pnls <- cbind(posv, pnls)
+   colnames(pnls) <- c("positions", "pnls")
+   pnls
+ } # end sim_ema
```

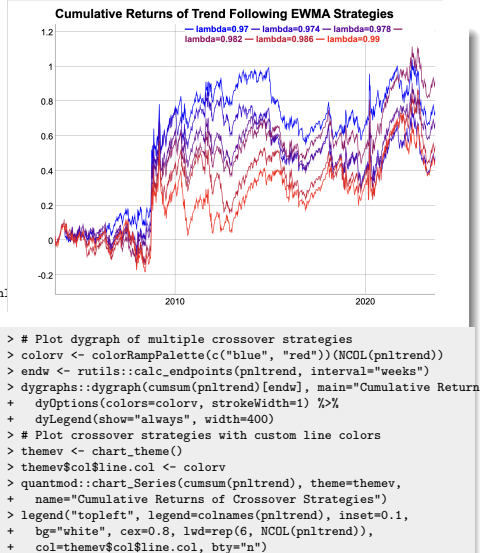
Simulating Multiple Trend Following Crossover Strategies

Multiple *EMA* strategies can be simulated by calling the function `sim_ema()` in a loop over a vector of λ parameters.

But `sim_ema()` returns an *xts* time series, and `apply()` cannot merge *xts* time series together.

So instead the loop is performed using `lapply()` which returns a list of *xts*, and the list is merged into a single *xts* using the functions `do.call()` and `cbind()`.

```
> lambdav <- seq(from=0.97, to=0.99, by=0.004)
> # Perform lapply() loop over lambdav
> pnltrend <- lapply(lambdav, function(lambdaf) {
+   # Simulate crossover strategy and calculate the returns
+   sim_ema(closep=closep, lambdaf=lambdaf, bidask=0, lag=2)[, "pn"]
+ }) # end lapply
> pnltrend <- do.call(cbind, pnltrend)
> colnames(pnltrend) <- paste0("lambda=", lambdav)
```



Simulating Crossover Strategies Using Parallel Computing

Simulating *EMA* strategies naturally lends itself to parallel computing, since the simulations are independent from each other.

The function `parLapply()` is similar to `lapply()`, and performs loops under *Windows* using parallel computing on several CPU cores.

The resulting list of time series can then be collapsed into a single *xts* series using the functions `rutils::do_call()` and `cbind()`.

```
> # Initialize compute cluster under Windows
> library(parallel)
> ncores <- detectCores() - 1 # Number of cores
> compclust <- makeCluster(detectCores()-1)
> clusterExport(compclust,
+   varlist=c("ohlcv", "lookb", "sim_ema"))
> # Perform parallel loop over lambdav under Windows
> pnltrend <- parLapply(compclust, lambdav, function(lambdaf) {
+   library(quantmod)
+   # Simulate crossover strategy and calculate the returns
+   sim_ema(closep=closep, lambdaf=lambdaf, bidask=0, lagg=2)[, "pnltrend"]
+ }) # end parLapply
> stopCluster(compclust) # Stop R processes over cluster under Windows
> # Perform parallel loop over lambdav under Mac-OSX or Linux
> pnltrend <- mclapply(lambdav, function(lambdaf) {
+   library(quantmod)
+   # Simulate crossover strategy and calculate the returns
+   sim_ema(closep=closep, lambdaf=lambdaf, bidask=0, lagg=2)[, "pnltrend"]
+ }, mc.cores=ncores) # end mclapply
> pnltrend <- do.call(cbind, pnltrend)
> colnames(pnltrend) <- paste0("lambda=", lambdav)
```

Optimal Decay Factor of Trend Following Crossover Strategies

The performance of trend following *EMA* strategies depends on the λ decay factor, with larger λ parameters closer to 1 performing better than larger ones.

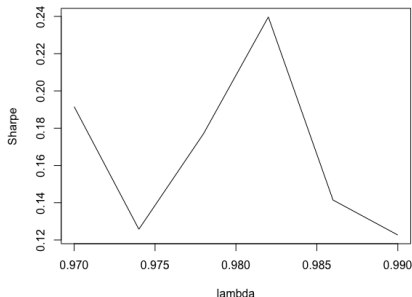
The optimal λ parameter applies significant weight to returns 8 – 12 months in the past, which is consistent with research on trend following strategies.

The *Sharpe ratios* of *EMA* strategies with different λ parameters can be calculated by performing an `sapply()` loop over the *columns* of returns.

`sapply()` treats the columns of *xts* time series as list elements, and loops over the columns.

Performing loops in R over the *columns* of returns is acceptable, but R loops over the *rows* of returns should be avoided.

Performance of EWMA Trend Following Strategies as Function of the Decay Parameter Lambda

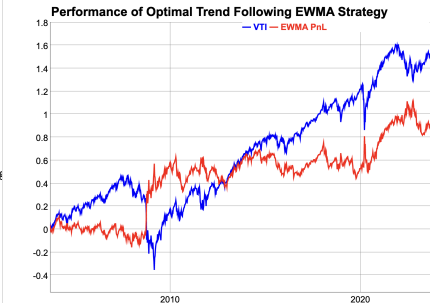


```
> # Calculate the annualized Sharpe ratios of strategy returns
> sharpetrend <- sqrt(252)*sapply(pnltrend, function(xtsv) {
+   mean(xtsv)/sd(xtsv)
+ }) # end sapply
> # Plot Sharpe ratios
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> plot(x=lambdav, y=sharpetrend, t="l",
+   xlab="lambda", ylab="Sharpe",
+   main="Performance of EMA Trend Following Strategies
+   as Function of the Decay Factor Lambda")
```

Optimal Trend Following Crossover Strategy

The best performing trend following *EMA* strategy has a relatively small λ parameter, corresponding to slower weight decay (giving more weight to past price), and producing less frequent trading.

```
> # Calculate the optimal lambda
> lambdaf <- lambdav[which.max(sharpetrend)]
> # Simulate best performing strategy
> ematrend <- sim_ema(closep=closep, lambdaf=lambdaf, bidask=0, lag=
> posv <- ematrend[, "positions"]
> trendopt <- ematrend[, "pnls"]
> wealthv <- cbind(retp, trendopt)
> colnames(wealthv) <- c("VTI", "EMA PnL")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> cor(wealthv)[1, 2]
> # Plot dygraph of crossover strategy wealth
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Performance of Op
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



```
> # Plot EMA PnL with position shading
> # Standard plot of crossover strategy wealth
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(cumsum(wealthv), theme=themev,
+   name="Performance of Crossover Strategy")
> add_TA(posv > 0, on=-1, col="lightgreen", border="lightgreen")
> add_TA(posv < 0, on=-1, col="lightgrey", border="lightgrey")
> legend("top", legend=colnames(wealthv),
+   inset=0.05, bg="white", lty=1, lwd=6,
+   col=themev$col$line.col, bty="n")
```

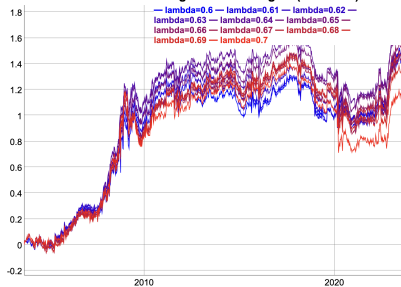
Mean Reverting EMA Crossover Strategies

Mean reverting EMA crossover strategies can be simulated using function `sim_ema()` with argument `trend=(-1)`.

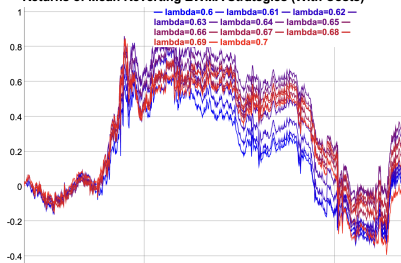
The profitability of mean reverting strategies can be significantly improved by using limit orders, to reduce transaction costs.

```
> lambdav <- seq(0.6, 0.7, 0.01)
> # Perform lapply() loop over lambdav
> pnlrevert <- lapply(lambdav, function(lambdaf) {
+   # Simulate crossover strategy and calculate the returns
+   sim_ema(closep=closep, lambdaf=lambdaf, bidask=0, trend=(-1))[,
+ }) # end lapply
> pnlrevert <- do.call(cbind, pnlrevert)
> colnames(pnlrevert) <- paste0("lambda=", lambdav)
> # Plot dygraph of mean reverting crossover strategies
> colorv <- colorRampPalette(c("blue", "red"))(NROW(lambdav))
> dygraphs::dygraph(cumsum(pnlrevert)[endw], main="Returns of Mean I
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=400)
> # Plot crossover strategies with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(pnlrevert,
+   theme=themev, name="Cumulative Returns of Mean Reverting Crosso
> legend("topleft", legend=colnames(pnlrevert),
+   inset=0.1, bg="white", cex=0.8, lwd=6,
+   col=themev$col$line.col, bty="n")
```

Returns of Mean Reverting EWMA Strategies (No Costs)



Returns of Mean Reverting EWMA Strategies (With Costs)



Performance of Mean Reverting Crossover Strategies

The *Sharpe ratios* of *EMA* strategies with different λ parameters can be calculated by performing an `apply()` loop over the *columns* of returns.

`apply()` treats the columns of *xts* time series as list elements, and loops over the columns.

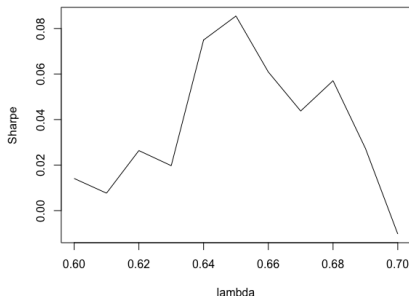
Performing loops in R over the *columns* of returns is acceptable, but R loops over the *rows* of returns should be avoided.

The performance of mean reverting *EMA* strategies depends on the λ parameter, with performance decreasing for very small or very large λ parameters.

For too large λ parameters, the trading frequency is too high, causing high transaction costs.

For too small λ parameters, the trading frequency is too low, causing the strategy to miss profitable trades.

Performance of EWMA Mean Reverting Strategies as Function of the Decay Parameter Lambda



```
> # Calculate the Sharpe ratios of strategy returns
> sharperevert <- sqrt(252)*apply(pnlrevert, function(xtsv) {
+   mean(xtsv)/sd(xtsv)
+ }) # end apply
> plot(x=lambdav, y=sharperevert, t="l",
+       xlab="lambda", ylab="Sharpe",
+       main="Performance of EMA Mean Reverting Strategies
+       as Function of the Decay Factor Lambda")
```

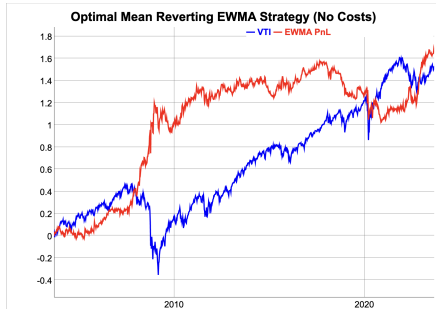
Optimal Mean Reverting Crossover Strategy

Reverting the direction of the trend following *EMA* strategy creates a mean reverting strategy.

The best performing mean reverting *EMA* strategy has a relatively large λ parameter, corresponding to faster weight decay (giving more weight to recent prices), and producing more frequent trading.

But a too large λ parameter also causes very high trading frequency, and high transaction costs.

```
> # Calculate the optimal lambda
> lambdaf <- lambdav[which.max(sharperevert)]
> # Simulate best performing strategy
> emarevert <- sim_ema(closep=closep, bidask=0.0,
+   lambdaf=lambdaf, trend=(-1))
> posv <- emarevert[, "positions"]
> revertopt <- emarevert[, "pnls"]
> wealthv <- cbind(retp, revertopt)
> colnames(wealthv) <- c("VTI", "EMA PnL")
> # Plot dygraph of crossover strategy wealth
> colorv <- c("blue", "red")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Optimal Mean Revert:
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```



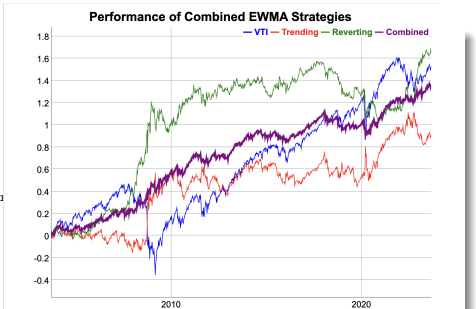
```
> # Standard plot of crossover strategy wealth
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(cumsum(wealthv), theme=themev,
+   name="Optimal Mean Reverting Crossover Strategy")
> add_TA(posv > 0, on=-1, col="lightgreen", border="lightgreen")
> add_TA(posv < 0, on=-1, col="lightgrey", border="lightgrey")
> legend("top", legend=colnames(wealthv),
+   inset=0.05, bg="white", lty=1, lwd=6,
+   col=themev$col$line.col, bty="n")
```

Combining Trend Following and Mean Reverting Strategies

The returns of trend following and mean reverting strategies are usually negatively correlated to each other, so combining them can achieve significant diversification of risk.

The main advantage of EMA crossover strategies is that they provide positive returns and a diversification of risk with respect to static stock portfolios.

```
> # Calculate the correlation between trend following and mean reverting
> trendopt <- ematrend[, "pnls"]
> colnames(trendopt) <- "trend"
> revertopt <- emarevert[, "pnls"]
> colnames(revertopt) <- "revert"
> cor(cbind(retp, trendopt, revertopt))
> # Calculate the combined strategy
> combstrat <- (retp + trendopt + revertopt)/3
> colnames(combstrat) <- "combined"
> # Calculate the annualized Sharpe ratio of strategy returns
> retc <- cbind(retp, trendopt, revertopt, combstrat)
> colnames(retc) <- c("VTI", "Trending", "Reverting", "Combined")
> sqrt(252)*apply(retc, function(xtsv) mean(xtsv)/sd(xtsv))
```



```
> # Plot dygraph of crossover strategy wealth
> colorv <- c("blue", "red", "green", "purple")
> dygraphs::dygraph(cumsum(retc)[endw], main="Performance of Combined EWMA Strategies")
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dySeries(name="Combined", strokeWidth=3) %>%
+ dyLegend(show="always", width=300)
> # Standard plot of crossover strategy wealth
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(pnls, theme=themev,
+ name="Performance of Combined Crossover Strategies")
> legend("topleft", legend=colnames(pnls),
+ inset=0.05, bg="white", lty=1, lwd=6,
+ col=themev$col$line.col, bty="n")
```

Ensemble of Crossover Strategies

Instead of selecting the best performing *EMA* strategy, one can choose a weighted average of strategies (ensemble), which corresponds to allocating positions according to the weights.

The weights can be chosen to be proportional to the Sharpe ratios of the *EMA* strategies.

```
> # Calculate the weights proportional to Sharpe ratios
> weightv <- c(sharpetrend, sharperevert)
> weightv[weightv < 0] <- 0
> weightv <- weightv/sum(weightv)
> retc <- cbind(pnltrrend, pnlrevert)
> retc <- retc %*% weightv
> retc <- cbind(retp, retc)
> colnames(retc) <- c("VTI", "EMA PnL")
> # Plot dygraph of crossover strategy wealth
> colorv <- c("blue", "red")
> dygraphs::dygraph(cumsum(retc)[endw], main="Performance of Ensemble of Crossover Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
> # Standard plot of crossover strategy wealth
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(cumsum(retc), theme=themev,
+   name="Performance of Ensemble of Crossover Strategies")
> legend("topleft", legend=colnames(pnls),
+   inset=0.05, bg="white", lty=1, lwd=6,
+   col=themev$col$line.col, bty="n")
```

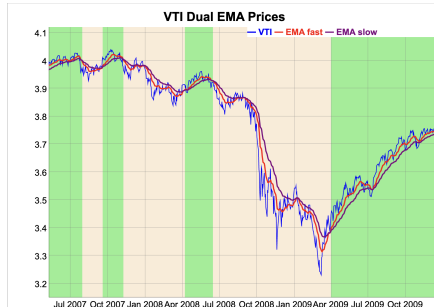


Dual EMA Crossover Strategy

In the *Dual EMA Crossover* strategy, the stock position depends on the difference between two moving averages.

The stock position flips when the fast moving *EMA* crosses the slow moving *EMA*.

```
> # Extract the log VTI prices
> ohlc <- log(rutils::etfenv$VTI)
> datev <- zoo::index(ohlc)
> nrow <- NROW(ohlc)
> closep <- quantmod::Cl(ohlc)
> colnames(closep) <- "VTI"
> retp <- rutils::diffit(closep) # VTI returns
> # Calculate the fast and slow EMAs
> lambdafa <- 0.89
> lambdasl <- 0.95
> # Calculate the EMA prices
> emaf <- HighFreq::run_mean(closep, lambda=lambdafa)
> emas <- HighFreq::run_mean(closep, lambda=lambdasl)
> pricev <- cbind(closep, emaf, emas)
> colnames(pricev) <- c("VTI", "EMA fast", "EMA slow")
> # Determine the positions and the trade dates right after the EM.
> indic <- sign(emaf - emas)
> posv <- rutils::lagit(indic)
> crossd <- (rutils::diffit(posv) != 0)
> # Create colors for background shading
> shadev <- posv[crossd]
> shadev <- ifelse(shadev == 1, "lightgreen", "antiquewhite")
> crossd <- c(datev[crossd], end(closep))
```



```
> # Plot dygraph
> colv <- colnames(pricev)
> dyplot <- dygraphs::dygraph(pricev, main="VTI Dual EMA Prices") %>%
+   dySeries(name=colv[1], strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="red") %>%
+   dySeries(name=colv[3], strokeWidth=2, col="purple") %>%
+   dyLegend(show="always", width=300)
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> dyplot
```

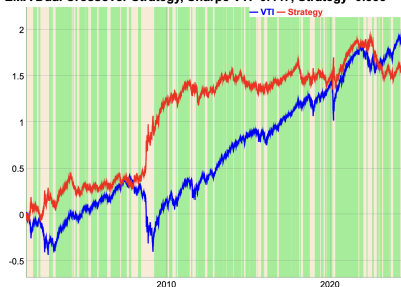
Dual EMA Crossover Strategy Performance

The crossover strategy suffers losses when prices are range-bound without a trend, because whenever it switches position the prices soon change direction. (This is called a "*whipsaw*".)

The crossover strategy performance is worse than the underlying asset (*VTI*), but it has a negative correlation to it, which is very valuable when building a portfolio.

```
> # Calculate the daily profits and losses of strategy
> pnls <- retp*posv
> colnames(pnls) <- "Strategy"
> wealthv <- cbind(retp, pnls)
> # Annualized Sharpe ratio of dual crossover strategy
> sharper <- sqrt(252)*sapply(wealthv, function (x) mean(x)/sd(x))
> # The crossover strategy has a negative correlation to VTI
> cor(wealthv)[1, 2]
```

EMA Dual Crossover Strategy, Sharpe VTI=0.447, Strategy=0.338



```
> # Plot Dual crossover strategy
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main=paste("EMA Dual Crossover Strategy, Sharpe", paste(paste(na
+   dyOptions(colors=c("blue", "red"), strokeWidth=2)
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], c
+ } # end for
> # Plot the dygraph object
> dyplot
```

Simulation Function for the Dual EMA Crossover Strategy

The *Dual EMA* strategy can be simulated by a single function, which allows the analysis of its performance depending on its parameters.

The function `sim_ema2()` performs a simulation of the *Dual EMA* strategy, given an *OHLC* time series of prices, and two decay factors λ_1 and λ_2 .

The function `sim_ema2()` returns the *EMA* strategy positions and returns, in a two-column *xts* time series.

```
> sim_ema2 <- function(closep, lambdafa=0.1, lambdasl=0.01,
+                       bidask=0.001, trend=1, lagg=1) {
+   if (lambdafa >= lambdasl) return(NA)
+   retp <- rutils::diffit(closep)
+   nrow <- NROW(closep)
+   # Calculate the EMA prices
+   emaf <- HighFreq::run_mean(closep, lambda=lambdafa)
+   emas <- HighFreq::run_mean(closep, lambda=lambdasl)
+   # Calculate the positions, either: -1, 0, or 1
+   indic <- sign(emaf - emas)
+   if (lagg > 1) {
+     indic <- HighFreq::roll_sum(indic, lagg)
+     indic[1:lagg] <- 0
+   } # end if
+   posv <- rep(NA_integer_, nrow)
+   posv[1] <- 0
+   posv <- ifelse(indic == lagg, 1, posv)
+   posv <- ifelse(indic == (-lagg), -1, posv)
+   posv <- zoo::na.locf(posv, na.rm=FALSE)
+   posv <- xts::xts(posv, order.by=zoo::index(closep))
+   # Lag the positions to trade on next day
+   posv <- rutils::lagit(posv, lagg=1)
+   # Calculate the PnLs of strategy
+   pnls <- retp*posv
+   costv <- 0.5*bidask*abs(rutils::diffit(posv))
+   pnls <- (pnls - costv)
+   # Calculate the strategy returns
+   pnls <- cbind(posv, pnls)
+   colnames(pnls) <- c("positions", "pnls")
+   pnls
+ } # end sim_ema2
```

Dual Crossover Strategy Performance Matrix

Multiple *Dual EMA* strategies can be simulated by calling the function `sim_ema2()` in two loops over the vectors of λ parameters.

The function `outer()` calculates the values of a function over a grid spanned by two variables, and returns a matrix of function values.

The function `Vectorize()` performs an `apply()` loop over the arguments of a function, and returns a vectorized version of the function.

```
> lambdafv <- seq(from=0.85, to=0.99, by=0.01)
> lambdasv <- seq(from=0.85, to=0.99, by=0.01)
> # Calculate the Sharpe ratio of dual crossover strategy
> calc_sharpe <- function(closep, lambdafa, lambdasl, bidask, trend,
+   if (lambdafa >= lambdasl) return(NA)
+   pnls <- sim_ema2(closep=closep, lambdafa=lambdafa, lambdasl=lambdasl,
+     bidask=bidask, trend=trend, lagg=lagg)[, "pnls"]
+   sqrt(252)*mean(pnls)/sd(pnls)
+ } # end calc_sharpe
> # Vectorize calc_sharpe with respect to lambdafa and lambdasl
> calc_sharpe <- Vectorize(FUN=calc_sharpe,
+   vectorize.args=c("lambdafa", "lambdasl"))
> # Calculate the matrix of PnLs
> sharpem <- outer(lambdafv, lambdasv, FUN=calc_sharpe,
+   closep=closep, bidask=0.0, trend=1, lagg=1)
> # Or perform two apply() loops over lambda vectors
> sharpem <- sapply(lambdasv, function(lambdasl) {
+   sapply(lambdafv, function(lambdafa) {
+     if (lambdafa >= lambdasl) return(NA)
+     calc_sharpe(closep=closep, lambdafa=lambdafa,
+       lambdasl=lambdasl, bidask=0.0, trend=1, lagg=1)
+   }) # end sapply
+ }) # end sapply
> colnames(sharpem) <- lambdasv
> rownames(sharpem) <- lambdafv
```

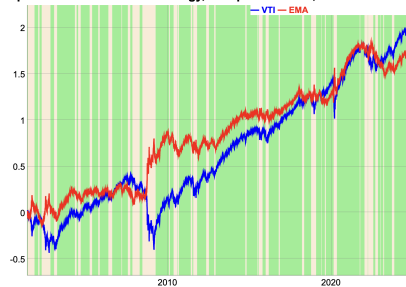

Optimal Dual Crossover Strategy

The best *Dual EMA* strategy performs better than the best *single EMA* strategy, because it has an extra parameter that can be adjusted to improve in-sample performance.

But this doesn't guarantee better out-of-sample performance.

```
> # Calculate the PnLs of the optimal crossover strategy
> whichv <- which(sharpem == max(sharpem, na.rm=TRUE), arr.ind=TRUE)
> lambdafa <- lambdafv[whichv[1]]
> lambdasl <- lambdasv[whichv[2]]
> crossopt <- sim_ema2(closep=closep, lambdafa=lambdafa, lambdasl=lambdasl,
+   bidask=0.0, trend=1, lag=1)
> pnls <- crossopt[, "pnls"]
> wealthv <- cbind(retp, pnls)
> colnames(wealthv)[2] <- "EMA"
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Annualized Sharpe ratio of dual crossover strategy
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
> # The crossover strategy has a negative correlation to VTI
> cor(wealthv)[1, 2]
```

Optimal Dual Crossover Strategy, Sharpe VTI=0.447, EMA=0.391



```
> # Create colors for background shading
> posv <- crossopt[, "positions"]
> crossd <- (rutils::diffit(posv) != 0)
> shadev <- posv[crossd]
> crossd <- c(zoo::index(shadev), end(posv))
> shadev <- ifelse(drop(zoo::coredata(shadev)) == 1, "lightgreen", "lightred")
> # Plot Optimal Dual crossover strategy
> dyplot <- dygraphs::dygraph(cumsum(wealthv), main=paste("Optimal Dual Crossover Strategy",
+   dyOptions(colors=c("blue", "red"), strokeWidth=2))
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], color=shadev[i])
+ } # end for
> # Plot the dygraph object
> dyplot
```

Performance of Dual Crossover Strategy Out-of-Sample

In-sample, the best *Dual EMA* strategy performs better than *VTI*, because it has two parameters that can be adjusted to improve performance.

But out-of-sample, the best *Dual EMA* strategy performs worse than *VTI*, because it's been *overfitted* in-sample.

```
> # Define in-sample and out-of-sample intervals
> insample <- 1:(nrows %/% 2)
> outsample <- (nrows %/% 2 + 1):nrows
> # Calculate the matrix of PnLs
> sharpem <- outer(lambdafv, lambdasv,
+ FUN=calc_sharpe, closep=closep[insample, ],
+ bidask=0.0, trend=1, lag=1)
> colnames(sharpem) <- lambdasv
> rownames(sharpem) <- lambdafv
> # Calculate the PnLs of the optimal strategy
> whichv <- which(sharpem == max(sharpem, na.rm=TRUE), arr.ind=TRUE)
> lambdafa <- lambdafv[whichv[1]]
> lambdasl <- lambdasv[whichv[2]]
> pnls <- sim_ema2(closep=closep, lambdafa=lambdafa, lambdasl=lamb
+ bidask=0.0, trend=1, lag=1)[, "pnls"]
> wealthv <- cbind(retsp, pnls)
> colnames(wealthv)[2] <- "EMA"
> # Calculate the Sharpe and Sortino ratios in-sample and out-of-sample
> sqrt(252)*sapply(wealthv[insample, ], function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> sqrt(252)*sapply(wealthv[outsample, ], function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



```
> # Dygraphs plot with custom line colors
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Dual Crossover Strategy")
+ dyEvent(zoo::index(wealthv[last(insample)]), label="in-sample",
+ dyOptions(colors=c("blue", "red"), strokeWidth=2)
```

Volume-Weighted Average Price Indicator

The Volume-Weighted Average Price (*VWAP*) is defined as the sum of prices multiplied by trading volumes, divided by the sum of volumes:

$$P_t^{VWAP} = \frac{\sum_{j=0}^n v_{t-j} p_{t-j}}{\sum_{j=0}^n v_{t-j}}$$

The *VWAP* applies more weight to prices with higher trading volumes, which allows it to react more quickly to recent market volatility.

The drawback of the *VWAP* indicator is that it applies large weights to prices far in the past.

The *VWAP* is often used as a technical indicator in trend following strategies.



```
> # Calculate the log OHLC prices and volumes
> ohlc <- rutils::etfenv$VTI
> closep <- log(quantmod::Cl(ohlc))
> colnames(closep) <- "VTI"
> volumv <- quantmod::Vo(ohlc)
> colnames(volumv) <- "Volume"
> nrow <- NROW(closep)
> # Calculate the VWAP prices
> lookb <- 21
> vwap <- HighFreq::roll_sum(closep, lookb=lookb, weightv=volumv)
> colnames(vwap) <- "VWAP"
> pricev <- cbind(closep, vwap)
```

```
> # Dygraphs plot with custom line colors
> colorv <- c("blue", "red")
> dygraphs::dygraph(pricev["2009"], main="VTI VWAP Prices") %>%
+   dyOptions(colors=colorv, strokeWidth=2)
> # Plot VWAP prices with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(pricev["2009"], theme=themev,
+   lwd=2, name="VTI VWAP Prices")
> legend("bottomright", legend=colnames(pricev),
+   inset=0.1, bg="white", lty=1, lwd=6, cex=0.8,
+   col=themev$col$line.col, bty="n")
```

Recursive VWAP Price Indicator

The VWAP prices p^{VWAP} can also be calculated as the ratio of the volume weighted prices μ^{PV} divided by the mean trading volumes μ^V :

$$p^{VWAP} = \frac{\mu^{PV}}{\mu^V}$$

The volume weighted prices μ^{PV} and the mean trading volumes μ^V are both calculated recursively:

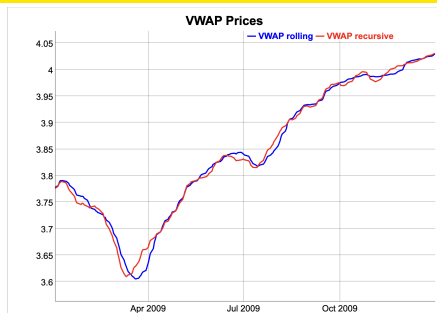
$$\begin{aligned}\mu_t^V &= \lambda \mu_{t-1}^V + (1 - \lambda) v_t \\ \mu_t^{PV} &= \lambda \mu_{t-1}^{PV} + (1 - \lambda) v_t p_t\end{aligned}$$

The recursive VWAP prices are slightly different from those calculated as a convolution, because the convolution uses a fixed look-back interval.

The advantage of the recursive VWAP indicator is that it gradually "forgets" about large trading volumes far in the past.

The compiled C++ function `stats::C_rfilter()` calculates the trailing weighted values recursively.

The function `HighFreq::run_mean()` also calculates the trailing weighted values recursively.



```
> # Calculate the VWAP prices recursively using C++ code
> lambdaf <- 0.9
> volumer <- .Call(stats::C_rfilter, volumv, lambdaf, c(as.numeric(volumv)))
> pricer <- .Call(stats::C_rfilter, volumv*closep, lambdaf, c(as.numeric(closep)))
> vwapr <- pricer/volumer
> # Calculate the VWAP prices recursively using RcppArmadillo
> vwapc <- HighFreq::run_mean(closep, lambda=lambdaf, weightv=volumv)
> all.equal(vwapr, drop(vwapc))
> # Dygraphs plot the VWAP prices
> pricev <- xts(cbind(vwapr, vwapr), zoo::index(ohlc))
> colnames(pricev) <- c("VWAP rolling", "VWAP recursive")
> dygraphs::dygraph(pricev["2009"], main="VWAP Prices") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Simulating the VWAP Crossover Strategy

In the trend following *VWAP Crossover* strategy, the stock position switches depending if the current price is above or below the *VWAP*.

If the current price crosses above the *VWAP*, then the strategy switches its stock position to a fixed unit of long risk, and if it crosses below, to a fixed unit of short risk.

To prevent whipsaws and over-trading, the crossover strategy delays switching positions until the indicator repeats the same value for several periods.

```
> # Calculate the VWAP prices recursively using RcppArmadillo
> lambdaf <- 0.99
> vwapc <- HighFreq::run_mean(closep, lambda=lambdaf, weightv=volum
> # Calculate the positions from lagged indicator
> indic <- sign(closep - vwapc)
> lagg <- 2
> indic <- HighFreq::roll_sum(indic, lagg)
> # Calculate the positions, either: -1, 0, or 1
> posv <- rep(NA_integer_, nrows)
> posv[1] <- 0
> posv <- ifelse(indic == lagg, 1, posv)
> posv <- ifelse(indic == (-lagg), -1, posv)
> posv <- zoo::na.locf(posv, na.rm=FALSE)
> posv <- xts::xts(posv, order.by=zoo::index(closep))
> # Lag the positions to trade in next period
> posv <- rutils::lagit(posv, lagg=1)
> # Calculate the PnLs of VWAP strategy
> retp <- rutils::diffit(closep) # VTI returns
> pnls <- retp*posv
> colnames(pnls) <- "VWAP"
> wealthv <- cbind(retp, pnls)
> colv <- colnames(wealthv)
```

VWAP Crossover Strategy, Sharpe VTI=0.399, VWAP=0.275



```
> # Annualized Sharpe ratios of VTI and VWAP strategy
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
> # Create colors for background shading
> crossd <- (rutils::diffit(posv) != 0)
> shadev <- posv[crossd]
> crossd <- c(zoo::index(shadev), end(posv))
> shadev <- ifelse(drop(zoo::coredata(shadev)) == 1, "lightgreen", "lightred")
> # Plot dygraph of VWAP strategy
> # Create dygraph object without plotting it
> dyplot <- dygraphs::dygraph(cumsum(wealthv),
+   main=paste("VWAP Crossover Strategy, Sharpe", paste(paste(names(wealthv)), sharper, sep=""),
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300))
> # Add shading to dygraph object
> for (i in 1:NROW(shadev)) {
+   dyplot <- dyplot %>% dyShading(from=crossd[i], to=crossd[i+1], color=shadev[i])
+ } # end for
> # Plot the dygraph object
```

Combining VWAP Crossover Strategy with Stocks

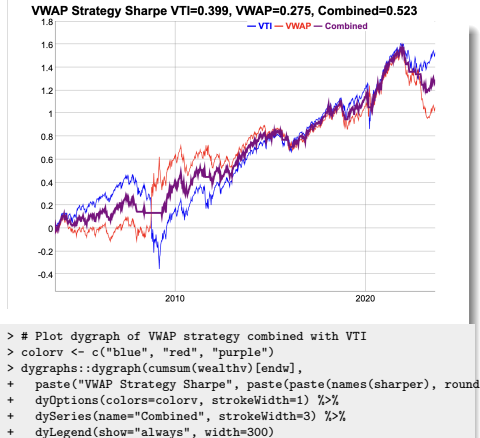
Even though the *VWAP* strategy doesn't perform as well as a static buy-and-hold strategy, it can provide risk reduction when combined with it.

This is because the *VWAP* strategy has a negative correlation with respect to the underlying asset.

In addition, the *VWAP* strategy performs well in periods of extreme market selloffs, so it can provide a hedge for a static buy-and-hold strategy.

The *VWAP* strategy serves as a dynamic put option in periods of extreme market selloffs.

```
> # Calculate the correlation of VWAP strategy with VTI
> cor(retp, pnls)
> # Combine VWAP strategy with VTI
> wealthv <- cbind(retp, pnls, 0.5*(retp+pnls))
> colnames(wealthv) <- c("VTI", "VWAP", "Combined")
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x))
```

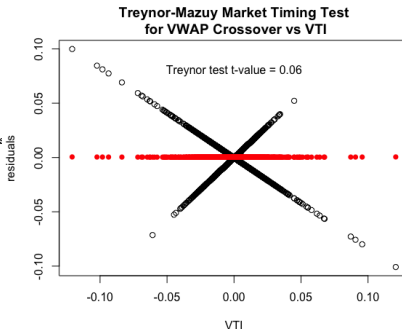


VWAP Crossover Strategy Market Timing Skill

The VWAP crossover strategy shorts the market during significant selloffs, but otherwise doesn't display market timing skill.

The t-value of the *Treynor-Mazuy* test is negative, but not statistically significant.

```
> # Test VWAP crossover market timing of VTI using Treynor-Mazuy test
> desm <- cbind(pnls, retp, retp^2)
> desm <- na.omit(desm)
> colnames(desm) <- c("VWAP", "VTI", "Treynor")
> regmod <- lm(VWAP ~ VTI + Treynor, data=desm)
> summary(regmod)
> # Plot residual scatterplot
> resid <- (desm$VWAP - regmod$coeff["VTI"]*retp)
> resid <- regmod$residuals
> # x11(width=6, height=6)
> plot.default(x=retp, y=resid, xlab="VTI", ylab="residuals")
> title(main="Treynor-Mazuy Market Timing Test\n for VWAP Crossover")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retp
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retp, y=fitv, pch=16, col="red")
> text(x=0.0, y=0.8*max(resid), paste("Treynor test t-value =", tvalue))
```



Simulation Function for VWAP Crossover Strategy

The *VWAP* strategy can be simulated by a single function, which allows the analysis of its performance depending on its parameters.

The function `sim_vwap()` performs a simulation of the *VWAP* strategy, given an *OHLC* time series of prices, and the length of the look-back interval (`lookb`).

The function `sim_vwap()` returns the *VWAP* strategy positions and returns, in a two-column *xts* time series.

```
> sim_vwap <- function(ohlc, lambdaf=0.9, bidask=0.001, trend=1, lag=1) {
+   closep <- log(quantmod::Cl(ohlc))
+   volumv <- quantmod::Vo(ohlc)
+   retp <- rutils::diffit(closep)
+   nrows <- NROW(ohlc)
+   # Calculate the VWAP prices
+   vwap <- HighFreq::run_mean(closep, lambda=lambdaf, weightv=volumv)
+   # Calculate the indicator
+   indic <- trend*sign(closep - vwap)
+   if (lagg > 1) {
+     indic <- HighFreq::roll_sum(indic, lagg)
+     indic[1:lagg] <- 0
+   } # end if
+   # Calculate the positions, either: -1, 0, or 1
+   posv <- rep(NA_integer_, nrows)
+   posv[1] <- 0
+   posv <- ifelse(indic == lagg, 1, posv)
+   posv <- ifelse(indic == (-lagg), -1, posv)
+   posv <- zoo::na.locf(posv, na.rm=FALSE)
+   posv <- xts::xts(posv, order.by=zoo::index(closep))
+   # Lag the positions to trade on next day
+   posv <- rutils::lagit(posv, lagg=1)
+   # Calculate the PnLs of strategy
+   pnls <- retp*posv
+   costv <- 0.5*bidask*abs(rutils::diffit(posv))
+   pnls <- (pnls - costv)
+   # Calculate the strategy returns
+   pnls <- cbind(posv, pnls)
+   colnames(pnls) <- c("positions", "pnls")
+   pnls
+ } # end sim_vwap
```

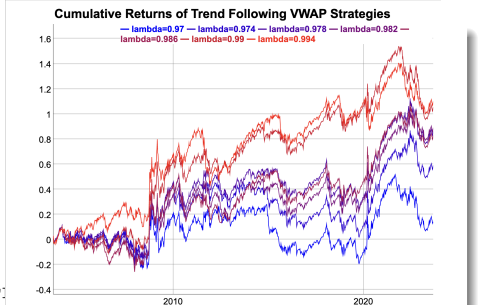

Simulating Multiple Trend Following VWAP Strategies

Multiple *VWAP* strategies can be simulated by calling the function `sim_vwap()` in a loop over a vector of λ parameters.

But `sim_vwap()` returns an *xts* time series, and `apply()` cannot merge *xts* time series together.

So instead the loop is performed using `lapply()` which returns a list of *xts*, and the list is merged into a single *xts* using the functions `do.call()` and `cbind()`.

```
> lambdav <- seq(from=0.97, to=0.995, by=0.004)
> # Perform lapply() loop over lambdav
> pnls <- lapply(lambdav, function(lambdaf) {
+   # Simulate VWAP strategy and Calculate the returns
+   sim_vwap(ohlc=ohlc, lambdaf=lambdaf, bidask=0, lagg=2)[, "pnls"]
+ }) # end lapply
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
```



```
> # Plot dygraph of multiple VWAP strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw], main="Cumulative Returns of
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dyLegend(show="always", width=500)
> # Plot VWAP strategies with custom line colors
> x11(width=6, height=5)
> themev <- chart_theme()
> themev$col$line.col <- colorv
> quantmod::chart_Series(cumsum(pnls), theme=themev,
+ name="Cumulative Returns of VWAP Strategies")
> legend("topleft", legend=colnames(pnls), inset=0.1,
+ bg="white", cex=0.8, lwd=rep(6, NCOL(pnls)),
+ col=themev$col$line.col, bty="n")
```

Backtesting of Rolling Strategies

Backtesting is the simulation of a trading strategy on historical data.

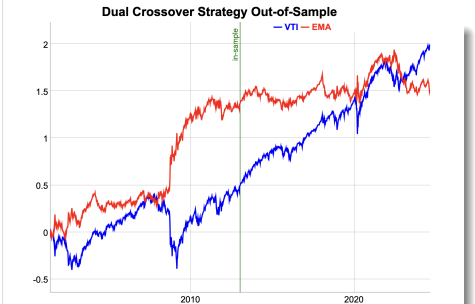
A *rolling strategy* can be *backtested* by specifying the parameter updating frequency, the formation interval, and the holding period:

- 1 Calculate the *end points* for parameter updating,
- 2 Define an objective function for parameter optimization,
- 3 Calculate the optimal parameters in the in-sample formation interval,
- 4 Calculate the out-of-sample strategy returns,
- 5 Calculate the transaction costs and subtract them from the strategy returns.

The *backtesting* redefines the problem of finding (tuning) the optimal trading strategy parameters, into the problem of finding the optimal *backtest* (meta-model) parameters: the updating frequency, the formation interval, and the holding period.

The advantage of using the *backtest* meta-model is that it can reduce the number of parameters that need to be optimized.

Using a different updating frequency in the *backtest* can produce different values for the optimal trading strategy parameters.



```
> # Dygraphs plot with custom line colors
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Dual Crossover Str
+   dyEvent(zoo::index(wealthv[last(insample)]), label="in-sample",
+   dyOptions(colors=c("blue", "red"), strokeWidth=2)
```

Bollinger Bands

The Bollinger Bands are three time series, with the middle band equal to the trailing mean prices, the upper band equal to the mean prices plus the trailing standard deviation, and the lower band equal to the mean prices minus the standard deviation.

The Bollinger Bands are often used to indicate that prices are cheap if they are below the lower band, and rich (expensive) if they are above the upper band.

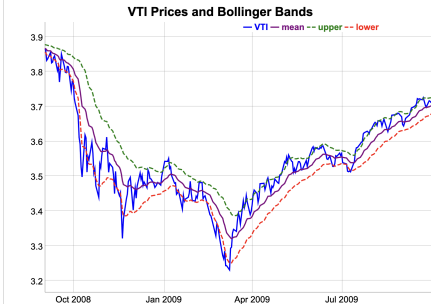
The function `HighFreq::run_var()` calculates the trailing mean and variance of the prices p_t , by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the prices minus the trailing means $(p_t - \bar{p}_t)^2$, using the decay factor λ :

$$\bar{p}_t = \lambda \bar{p}_{t-1} + (1 - \lambda) p_t$$

$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2) (p_t - \bar{p}_t)^2$$

Where $\bar{p}_t$ and σ_t^2 are the trailing mean and variance at time t .

The decay factor λ determines how quickly the mean and variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent prices, and vice versa.



```
> # Calculate the trailing mean prices and volatilities
> pricev <- log(na.omit(rutils::etfenv$prices$VTI))
> lambdaf <- 0.9
> volp <- HighFreq::run_var(pricev, lambda=lambdaf)
> pricema <- volp[, 1]
> volp <- sqrt(volp[, 2])
> # Dygraphs plot of Bollinger bands
> priceb <- cbind(pricev, pricema, pricema+volp, pricema-volp)
> colnames(priceb)[2:4] <- c("mean", "upper", "lower")
> colv <- colnames(priceb)
> dygraphs::dygraph(priceb["2008-09/2009-09"], main="VTI Prices and
+   dySeries(name=colv[1], strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="purple") %>%
+   dySeries(name=colv[3], strokeWidth=2, strokePattern="dashed", co
+   dySeries(name=colv[4], strokeWidth=2, strokePattern="dashed", co
+   dyLegend(show="always", width=300)
```

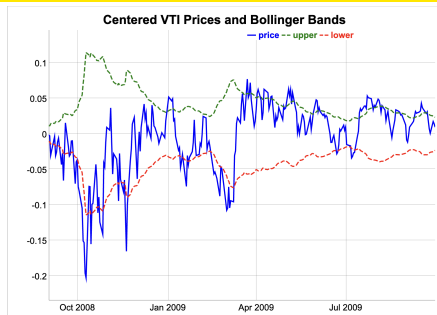
Bollinger Bands With Centered Prices

Centering (de-meaning) the prices provides a better illustration of the *Bollinger Bands*.

The centered prices tend to be range-bound between the *Bollinger Bands*.

When the centered price is below the lower band it's considered cheap, and if it's above the upper band it's considered rich (expensive).

When the centered price is close to zero it's considered fair (neutral).



```
> # Center the prices
> pricec <- pricev - pricema
> # Dygraphs plot of Bollinger bands
> priceb <- cbind(pricec, volp, -volp)
> colnames(priceb) <- c("price", "upper", "lower")
> colv <- colnames(priceb)
> dygraphs::dygraph(priceb["2008-09/2009-09"],
+   main="Centered VTI Prices and Bollinger Bands") %>%
+   dySeries(name=colv[1], strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, strokePattern="dashed", col="green") %>%
+   dySeries(name=colv[3], strokeWidth=2, strokePattern="dashed", col="red") %>%
+   dyLegend(show="always", width=300)
```

The Bollinger Band Strategy

The *Bollinger Strategy* switches to long \$1 dollar of the stock when prices are cheap (below the lower band), and sells short -\$1 dollar when prices are rich (expensive - above the upper band). It goes flat (unwinds) if the stock reaches a fair (mean) price.

The strategy is therefore always either long \$1 dollar of stock, or short -\$1 dollar, or flat the stock (\$0 dollars).

The upper and lower Bollinger bands can be chosen to be a multiple of the standard deviations above and below the mean prices.

The *Bollinger Strategy* is a *mean reverting* (contrarian) strategy because it bets on prices reverting to their mean value.

The *Bollinger Strategy* can be considered an extension of the crossover strategy, which utilizes information about the volatility of the returns.

The *Bollinger Strategy* switches its position only after the stock price crosses the Bollinger band, instead of the moving average price, which delays trades and reduces "whipsaws".

The *Bollinger Strategy* is a type of *statistical arbitrage* strategy.

Statistical arbitrage strategies try to exploit short-term anomalies in prices, when prices diverge from their equilibrium values and then revert back to them.

```
> # Calculate the EMA prices and volatilities
> lambdaf <- 0.1
> volp <- HighFreq::run_var(pricev, lambda=lambdaf)
> pricema <- volp[, 1]
> volp <- sqrt(volp[, 2])
> # Prepare the simulation parameters
> pricen <- as.numeric(pricev) # Numeric price
> pricec <- pricen - pricema # Centered price
> threshv <- volp
> nrows <- NROW(pricev)
> posv <- integer(nrows) # Stock positions
> posv[1] <- 0 # Initial position
> # Calculate the positions from Bollinger bands
> for (it in 2:nrows) {
+   if (pricec[it-1] > threshv[it-1]) {
+     # Enter short
+     posv[it] <- (-1)
+   } else if (pricec[it-1] < (-threshv[it-1])) {
+     # Enter long
+     posv[it] <- 1
+   } else if ((posv[it-1] < 0) && (pricec[it-1] < 0)) {
+     # Unwind short
+     posv[it] <- 0
+   } else if ((posv[it-1] > 0) && (pricec[it-1] > 0)) {
+     # Unwind long
+     posv[it] <- 0
+   } else {
+     # Do nothing
+     posv[it] <- posv[it-1]
+   } # end if
+ } # end for
> # Calculate the number of trades
> ntrades <- sum(abs(rutils::diffit(posv)) > 0)
> # Calculate the pnls
> retp <- rutils::diffit(pricev)
> pnls <- retp*posv
```

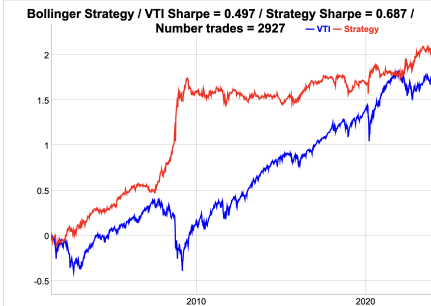
Bollinger Strategy Performance

The *Bollinger Band* strategy has two parameters: the decay factor λ and the standard deviation multiple n .

The best strategy parameters can be found using backtest simulation, but it risks overfitting the parameters to the in-sample data, and poor performance out-of-sample.

The *Bollinger Band* strategy has performed well for *VTI* with $\lambda = 0.1$, but it hasn't performed well for most other stocks.

The *Bollinger Band* strategy had its best performance for *VTI* prior to the financial crisis of 2008–2009.



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x[<0])
> sharper <- round(sharper, 3)
> # Dygraphs plot of Bollinger strategy
> colv <- colnames(wealthv)
> captiont <- paste("Bollinger Strategy", "/ \n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main=captiont) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

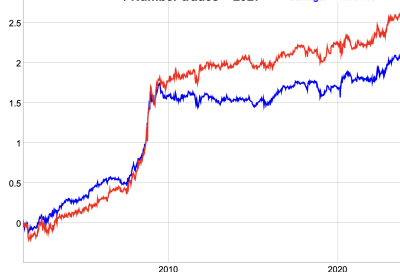
The Patient Bollinger Strategy

Unwinding the position when the stock reaches a fair (mean) price doesn't necessarily produce better performance.

In the patient Bollinger Strategy the positions are held until the price reaches the opposite extreme, so that the strategy is always either long \$1 dollar of stock or short \$1 dollar of stock.

```
> # Simulate the patient Bollinger Strategy
> posv <- integer(nrows) # Stock positions
> posv[1] <- 0 # Initial position
> for (it in 2:nrows) {
+   if (pricec[it-1] > thresvh[it-1]) {
+     # Enter short
+     posv[it] <- (-1)
+   } else if (pricec[it-1] < (-thresvh[it-1])) {
+     # Enter long
+     posv[it] <- 1
+   } else {
+     # Do nothing
+     posv[it] <- posv[it-1]
+   } # end if
+ } # end for
> # Calculate the PnLs
> pnl2 <- retp*posv
```

Bollinger Strategy / Bollinger Sharpe = 0.687 / Modified Sharpe = 0.833
/ Number trades = 2927 — Bollinger — Modified



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(pnl1, pnl2)
> colnames(wealthv) <- c("Bollinger", "Patient")
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x[1:1000]))
> sharper <- round(sharper, 3)
> # Dygraphs plot of Bollinger strategy
> colv <- colnames(wealthv)
> caption <- paste("Bollinger Strategy", "/ \n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw, main=caption]) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Fast Bollinger Strategy Simulation

The Bollinger Strategy is path-dependent so simulating it requires performing a loop, which can be slow in R.

The patient Bollinger Strategy can be simulated quickly using the compiled C++ functions `ifelse()` and `zoo::na.locf()`.

```
> # Simulate the patient Bollinger Strategy quickly
> posf <- rep(NA_integer_, nrow)
> posf[1] <- 0
> posf <- ifelse(pricec > threshv, -1, posf)
> posf <- ifelse(pricec < -threshv, 1, posf)
> posf <- zoo::na.locf(posf)
> # Lag the positions to trade in the next period
> posf <- rutils::lagit(posf, lagg=1)
> # Compare the positions
> all.equal(posv, posf)
```

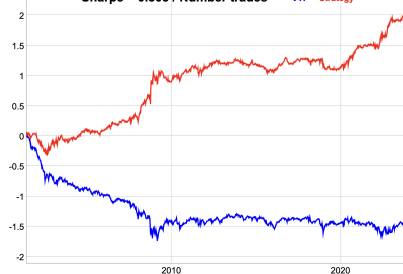

Bollinger Strategy For Daytime VTI Returns

The Bollinger Strategy has performed well for daytime returns of the VTI ETF, because daytime returns exhibit significant mean-reversion.

This simulation doesn't account for transaction costs, which would likely erase all profits if market orders were used for trade executions. But the strategy could be profitable if limit orders were used for trade executions.

```
> # Calculate the daytime open-to-close VTI returns
> ohlc <- log(rutils::etfenv$VTI)
> nrows <- NROW(ohlc)
> openp <- quantmod::Op(ohlc)
> highp <- quantmod::Hi(ohlc)
> lowp <- quantmod::Lo(ohlc)
> closep <- quantmod::Cl(ohlc)
> retc <- (closep - openp)
> # Calculate the cumulative daytime VTI returns
> priced <- cumsum(retc)
> lambdaf <- 0.1
> volp <- HighFreq::run_var(priced, lambda=lambdaf)
> pricema <- volp[, 1]
> volp <- sqrt(volp[, 2])
> # Calculate the positions from Bollinger bands
> threshv <- volp
> pricec <- zoo::coredata(priced - pricema)
> posv <- rep(NA_integer_, nrow(priced))
> posv[1] <- 0
> posv <- ifelse(pricec > threshv, -1, posv)
> posv <- ifelse(pricec < -threshv, 1, posv)
> posv <- zoo::na.locf(posv)
> # Lag the positions to trade in the next period
> posv <- rutils::lagit(posv, lag=1)
> # Calculate the number of trades and the PnLs
> ntrades <- sum(abs(rutils::diffit(posv)) > 0)
> pnls <- retc*posv
```

Bollinger Strategy for Daytime VTI / VTI Sharpe = -0.546 / Strategy Sharpe = 0.803 / Number trades = 12



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(retc, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> nyears <- as.numeric(end(priced)-start(priced))/365
> sharper <- sqrt(nrows/nyears)*sapply(wealthv, function(x) mean(x))
> sharper <- round(sharper, 3)
> # Dygraphs plot of Bollinger strategy
> colv <- colnames(wealthv)
> captioent <- paste("Bollinger Strategy for Daytime VTI", "/ \n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main=captioent) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Bollinger Strategy For Intraday SPY Returns

The Bollinger Strategy has performed well for 1-minute prices of the *SPY* ETF, because intraday returns exhibit significant mean-reversion.

This simulation doesn't account for transaction costs, which would likely erase all profits if market orders were used for trade executions. But the strategy could be profitable if limit orders were used for trade executions.

```
> # Calculate the trailing mean prices and volatilities of SPY
> pricev <- log(quantmod::Cl(HighFreq::SPY))
> nrows <- NROW(pricev)
> lambdaf <- 0.1
> volp <- HighFreq::run_var(pricev, lambda=lambdaf)
> pricema <- volp[, 1]
> volp <- sqrt(volp[, 2])
> # Calculate the positions from Bollinger bands
> threshv <- volp
> pricec <- zoo::coredata(pricev - pricema)
> posv <- rep(NA_integer_, nrows)
> posv[1] <- 0
> posv <- ifelse(pricec > threshv, -1, posv)
> posv <- ifelse(pricec < -threshv, 1, posv)
> posv <- zoo::na.locf(posv)
> # Lag the positions to trade in the next period
> posv <- rutils::lagit(posv, lag=1)
> # Calculate the number of trades and the PnLs
> ntrades <- sum(abs(rutils::diffit(posv)) > 0)
> retp <- rutils::diffit(pricev)
> pnls <- retp*posv
> # Subtract transaction costs from the pnls
> bidask <- 0.0001 # Bid-ask spread equal to 1 basis point
> costv <- 0.5*bidask*abs(rutils::diffit(posv))
> pnls <- (pnls - costv)
```

Bollinger Strategy for Minute SPY / SPY Sharpe = 0.189 / Strategy Sharpe = 1.417 / Number trades = 132285



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("SPY", "Strategy")
> nyyears <- as.numeric(end(pricev)-start(pricev))/365
> sharper <- sqrt(nrows/nyyears)*sapply(wealthv, function(x) mean(x))
> sharper <- round(sharper, 3)
> # Dygraphs plot of Bollinger strategy
> colv <- colnames(wealthv)
> caption <- paste("Bollinger Strategy for Minute SPY", "/ \n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw, main=caption]) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=100)
```

The Hampel Filter Bands

The Bollinger bands can be improved by using nonparametric measures of location (*median*) and dispersion (*MAD*).

The *Median Absolute Deviation* (*MAD*) is a nonparametric measure of dispersion (variability):

$$MAD = \text{median}(\text{abs}(p_t - \text{median}(\mathbf{p})))$$

The *Hampel z-score* is equal to the deviation from the median divided by the *MAD*:

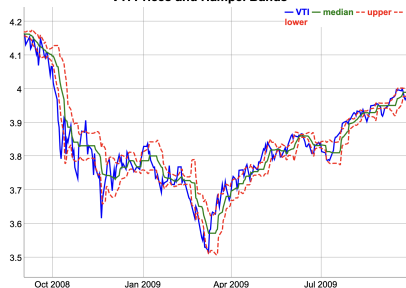
$$z_i = \frac{p_t - \text{median}(\mathbf{p})}{MAD}$$

A time series of *z-scores* over past data can be calculated using a trailing look-back window.

```
> # Extract time series of VTI log prices
> pricev <- log(na.omit(rutils::etfenv$prices$VTI))
> nrow <- NROW(pricev)
> # Define look-back window
> lookb <- 11
> # Calculate time series of trailing medians
> medianv <- HighFreq::roll_mean(pricev, lookb, method="nonparamet")
> # medianv <- TTR::runMedian(pricev, n=lookb)
> # Calculate time series of MAD
> madv <- HighFreq::roll_var(pricev, lookb=lookb, method="nonparam")
> # madv <- TTR::runMAD(pricev, n=lookb)
> # Calculate time series of z-scores
> zscores <- ifelse(madv > 0, (pricev - medianv)/madv, 0)
> zscores[1:lookb, ] <- 0
> tail(zscores, lookb)
> range(zscores)
```

```
> # Plot histogram of z-scores
> histp <- hist(zscores, col="lightgrey",
+   xlab="z-scores", breaks=50, xlim=c(-4, 4),
+   ylab="frequency", freq=FALSE, main="Hampel Z-Scores histogram")
> lines(density(zscores, adjust=1.5), lwd=3, col="blue")
> # Dygraphs plot of Hampel bands
> priceb <- cbind(pricev, medianv, medianv+madv, medianv-madv)
> colnames(priceb)[2:4] <- c("median", "upper", "lower")
> colv <- colnames(priceb)
> dygraphs::dygraph(priceb["2008-09/2009-09"], main="VTI Prices and
+   dySeries(name=colv[1], strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, col="green") %>%
+   dySeries(name=colv[3], strokeWidth=2, strokePattern="dashed", col="red") %>%
+   dySeries(name=colv[4], strokeWidth=2, strokePattern="dashed", col="red") %>%
+   dyLegend(show="always", width=300)
```

VTI Prices and Hampel Bands



Hampel Filter Strategy

The Hampel filter strategy is a contrarian strategy that uses Hampel z-scores to establish long and short positions.

The Hampel strategy has two meta-parameters: the look-back interval and the threshold level.

The best choice of the meta-parameters can be determined through simulation.

```
> # Calculate the time series of trailing medians and MAD
> lookb <- 3
> medianv <- HighFreq::roll_mean(pricev, lookb, method="nonparametric")
> madv <- HighFreq::roll_var(pricev, lookb=lookb, method="nonparametric")
> # Calculate the time series of z-scores
> zscores <- ifelse(madv > 0, (pricev - medianv)/madv, 0)
> zscores[1:lookb, ] <- 0
> range(zscores)
> # Calculate the positions
> threshv <- 1
> posv <- rep(NA_integer_, nrow)
> posv[1] <- 0
> posv[zscores > threshv] <- (-1)
> posv[zscores < -threshv] <- 1
> posv <- zoo::na.locf(posv)
> posv <- rutils::lagit(posv)
> # Calculate the number of trades and the PnLs
> ntrades <- sum(abs(rutils::diffit(posv)) > 0)
> retp <- rutils::diffit(pricev)
> pnls <- retp*posv
```

Hampel Strategy / VTI Sharpe = 0.497 / median Sharpe = 0.705 /
Number trades = 903



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> sharper <- sqrt(252)*sapply(wealthv, function(x) mean(x)/sd(x)*100)
> sharper <- round(sharper, 3)
> # Dygraphs plot of Hampel strategy
> captio <- paste("Hampel Strategy", "\n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> colv <- colnames(wealthv)
> dygraphs::dygraph(cumsum(wealthv)[endw], main=captio) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=100)
```

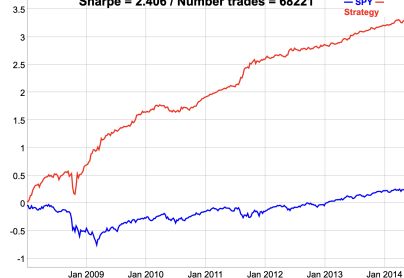
Hampel Filter Strategy For Intraday SPY Returns

The Hampel Filter strategy has performed well for 1-minute prices of the *SPY* ETF, because intraday returns exhibit significant mean-reversion.

This simulation doesn't account for transaction costs, which would likely erase all profits if market orders were used for trade executions. But the strategy could be profitable if limit orders were used for trade executions.

```
> # Calculate the trailing mean prices and volatilities of SPY
> pricev <- log(quantmod::Cl(HighFreq::SPY))
> nrows <- NROW(pricev)
> # Calculate the price medians and MAD
> lookb <- 3
> medianv <- HighFreq::roll_mean(pricev, lookb, method="nonparametric")
> madv <- HighFreq::roll_var(pricev, lookb=lookb, method="nonparametric")
> # Calculate the time series of z-scores
> zscores <- ifelse(madv > 0, (pricev - medianv)/madv, 0)
> zscores[1:lookb, ] <- 0
> # Calculate the positions
> threshv <- 1
> posv <- rep(NA_integer_, nrows)
> posv[1] <- 0
> posv[zscores < -threshv] <- 1
> posv[zscores > threshv] <- (-1)
> posv <- zoo::na.locf(posv)
> posv <- rutils::lagit(posv)
> # Calculate the number of trades and the PnLs
> ntrades <- sum(abs(rutils::diffit(posv)) > 0)
> retp <- rutils::diffit(pricev)
> pnls <- retp*posv
> # Subtract transaction costs from the pnls
> costv <- 0.5*bidask*abs(rutils::diffit(posv))
> pnls <- (pnls - costv)
```

Hampel Strategy for Minute SPY / SPY Sharpe = 0.189 / Strategy Sharpe = 2.406 / Number trades = 68221



```
> # Calculate the Sharpe ratios
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("SPY", "Strategy")
> nyears <- as.numeric(end(pricev)-start(pricev))/365
> sharper <- sqrt(nrows/nyears)*sapply(wealthv, function(x) mean(x)/sd(x))
> sharper <- round(sharper, 3)
> # Dygraphs plot of Hampel strategy
> colv <- colnames(wealthv)
> caption <- paste("Hampel Strategy for Minute SPY", "\n",
+   paste0(paste(colv[1:2], "Sharpe =", sharper), collapse=" / "),
+   "Number trades =", ntrades)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw, main=caption]) %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=100)
```

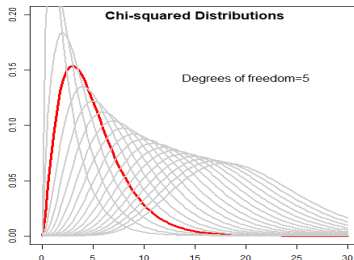
Plotting Using Expression Objects

It's sometimes convenient to create an *expression* object containing plotting commands, to be able to later create plots using it.

The function `quote()` produces an *expression* object without evaluating it.

The function `eval()` evaluates an *expression* in a specified *environment*.

```
> # Create a plotting expression
> expv <- quote({
+   degf <- 2:20
+   rangev <- (1:NROW(degf))
+   indeks <- 4
+   # Plot a curve
+   curve(expr=dchisq(x, df=degf[indeks]),
+   xlim=c(0, 30), ylim=c(0, 0.2),
+   xlab="", ylab="", lwd=3, col="red")
+   # Add grey lines to plot
+   for (it in rangev[-indeks]) {
+     curve(expr=dchisq(x, df=degf[it]),
+     xlim=c(0, 30), ylim=c(0, 0.2),
+     xlab="", ylab="", lwd=2, col="grey80", add=TRUE)
+   } # end for
+   # Add title
+   title(main="Chi-squared Distributions", line=-1.5, cex.main=1.5)
+   # Add legend
+   text(x=20, y=0.15, labels=paste0("Degrees of freedom=",
+   degf[indeks]), pos=1, cex=1.3)
+ }) # end quote
```



```
> # View the plotting expression
> expv
> # Create plot by evaluating the plotting expression
> x11(width=6, height=4)
> eval(expv)
```

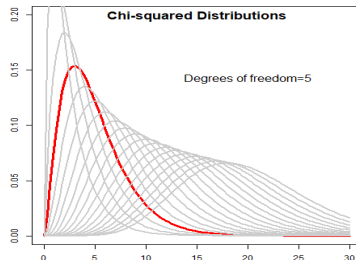
Animated Plots Using Package *animation*

The package *animation* allows creating animated plots in the form of *gif* and *html* documents.

The function `saveGIF()` produces a *gif* image with an animated plot.

The function `saveHTML()` produces an *html* document with an animated plot.

```
> library(animation)
> # Create an expression for creating multiple plots
> expv <- quote({
+   degf <- 2:20
+   rangev <- (1:NROW(degf))
+   # Set image refresh interval
+   animation::ani.options(interval=0.5)
+   # Create multiple plots with curves
+   for (indeks in rangev) {
+     curve(expr=dchisq(x, df=degf[indeks]),
+     xlim=c(0, 30), ylim=c(0, 0.2),
+     xlab="", ylab="", lwd=3, col="red")
+     # Add grey lines to plot
+     for (it in rangev[-indeks]) {
+       curve(expr=dchisq(x, df=degf[it]),
+       xlim=c(0, 30), ylim=c(0, 0.2),
+       xlab="", ylab="", lwd=2, col="grey80", add=TRUE)
+     } # end for
+     # Add title
+     title(main="Chi-squared Distributions", line=-1.5, cex.main=
+     # Add legend
+     text(x=20, y=0.15, labels=paste0("Degrees of freedom=",
+     degf[indeks]), pos=1, cex=1.3)
+   } # end for
+ }) # end quote
```



```
> # Create plot by evaluating the plotting expression
> x11(width=6, height=4)
> eval(expv)
> # Create gif with animated plot
> animation::saveGIF(expr=eval(expv),
+   movie.name="chi_squared.gif",
+   img.name="chi_squared")
> # Create html with animated plot
> animation::saveHTML(expr=eval(expv),
+   img.name="chi_squared",
+   htmlfile="chi_squared.html",
+   description="Chi-squared Distributions") # end saveHTML
```

Dynamic Documents Using *R* markdown

markdown is a simple markup language designed for creating documents in different formats, including *pdf* and *html*.

R Markdown is a modified version of *markdown*, which allows creating documents containing *math formulas* and *R* code embedded in them.

An *R Markdown* document (with extension *.Rmd*) contains:

- A *YAML* header,
- Text in *R Markdown* code format,
- Math formulas (equations), delimited using either single "\$" symbols (for inline formulas), or double "\$\$" symbols (for display formulas),
- *R* code chunks, delimited using either single "" backtick symbols (for inline code), or triple "" backtick symbols (for display code).

The packages *rmarkdown* and *knitr* compile *R* documents into either *pdf*, *html*, or *MS Word* documents.

```
---
title: "My First R Markdown Document"
author: Jerzy Pawlowski
date: 'r format(Sys.time(), "%m/%d/%Y")'
output: html_document
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)

install package quantmod if it can't be loaded success
if (!require("quantmod"))
 install.packages("quantmod")
```

### R Markdown
This is an *R Markdown* document. Markdown is a simple f

One of the advantages of writing documents *R Markdown*

You can read more about publishing documents using *R* h
https://algoquant.github.io/r,/markdown/2016/07/02/Publi

You can read more about using *R* to create *HTML* docum
https://algoquant.github.io/2016/07/05/Interactive-Plots

Clicking the Knit button in RStudio, compiles the

Example of an *R* code chunk:
```{r cars}
summary(cars)
```

### Plots in *R Markdown* documents

Plots can also be embedded, for example:
```{r pressure, echo=FALSE}
plot(pressure)
```
```


Package *shiny* for Creating Interactive Applications

The package *shiny* creates interactive applications running in R, with their outputs presented as live visualizations.

Shiny allows changing the model parameters, recalculating the model, and displaying the resulting outputs as plots and charts.

A *shiny app* is a file with *shiny* commands and R code.

The *shiny* code consists of a *shiny interface* and a *shiny server*.

The *shiny interface* contains widgets for data input and an area for plotting.

The *shiny server* contains the R model code and the plotting code.

The function `shiny::fluidPage()` creates a GUI layout for the user inputs of model parameters and an area for plots and charts.

The function `shiny::renderPlot()` renders a plot from the outputs of a live model.

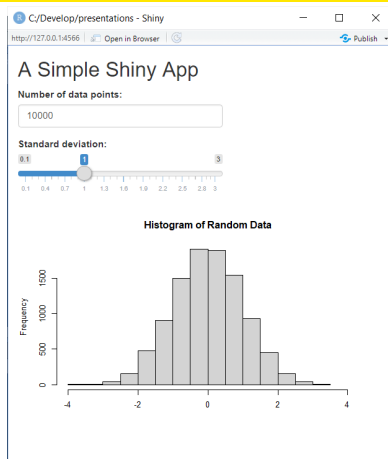
The function `shiny::shinyApp()` creates a shiny app from a *shiny interface* and a *shiny server*.

```
> ## App setup code that runs only once at startup.
> ndata <- 1e4
> stdev <- 1.0
>
> ## Define the user interface
> ui <- shiny::fluidPage(
+   # Create numeric input for the number of data points.
+   numericInput("ndata", "Number of data points:", value=ndata),
+   # Create slider input for the standard deviation parameter.
+   sliderInput("stdev", label="Standard deviation:",
+     min=0.1, max=3.0, value=stdev, step=0.1),
+   # Render plot in a panel.
+   plotOutput("plotobj", height=300, width=500)
+ ) # end user interface
>
> ## Define the server function
> servfun <- function(input, output) {
+   output$plotobj <- shiny::renderPlot({
+     # Simulate the data
+     datav <- rnorm(input$ndata, sd=input$stdev)
+     # Plot the data
+     par(mar=c(2, 4, 4, 0), oma=c(0, 0, 0, 0))
+     hist(datav, xlim=c(-4, 4), main="Histogram of Random Data")
+   }) # end renderPlot
+ } # end servfun
>
> # Return a Shiny app object
> shiny::shinyApp(ui=ui, server=servfun)
```

Running Shiny Apps in RStudio

A *shiny app* can be run by pressing the "Run App" button in RStudio.

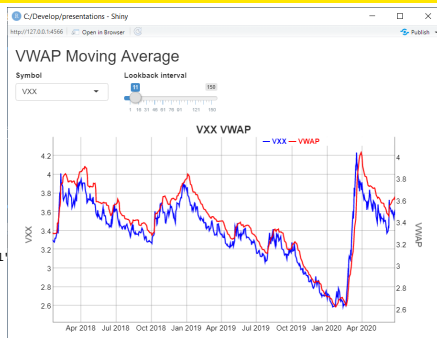
When the *shiny app* is run, the *shiny* commands are translated into *JavaScript* code, which creates a graphical user interface (GUI) with buttons, sliders, and boxes for data input, and also with the output plots and charts.



Positioning and Sizing Widgets Within the Shiny GUI

The functions `shiny::fluidRow()` and `shiny::column()` allow positioning and sizing widgets within the *shiny* GUI.

```
> ## Create elements of the user interface
> uiface <- shiny::fluidPage(
+   titlePanel("VWAP Moving Average"),
+   # Create single row of widgets with two slider inputs
+   fluidRow(
+     # Input stock symbol
+     column(width=3, selectInput("symbol", label="Symbol",
+                                   choices=symbolv, selected=symbol)),
+     # Input look-back interval
+     column(width=3, sliderInput("lookb", label="Lookback interval",
+                                   min=1, max=150, value=11, step=1))
+   ), # end fluidRow
+   # Create output plot panel
+   mainPanel(dygraphs::dygraphOutput("dyplot"), width=12)
+ ) # end fluidPage interface
```



Shiny Apps With Reactive Expressions

The package *shiny* allows specifying reactive expressions which are evaluated only when their input data is updated.

Reactive expressions avoid performing unnecessary calculations.

If the reactive expression is invalidated (recalculated), then other expressions that depend on its output are also recalculated.

This way calculations cascade through the expressions that depend on each other.

The function `shiny::reactive()` transforms an expression into a reactive expression.

```
> ## Define the server function
> servfun <- shiny::shinyServer(function(input, output) {
+   # Get the close and volume data in a reactive environment
+   closep <- shiny::reactive({
+     # Get the data
+     ohlc <- get(input$symbol, data_env)
+     closep <- log(quantmod::Cl(ohlc))
+     volum <- quantmod::Vo(ohlc)
+     # Return the data
+     cbind(closep, volum)
+   }) # end reactive code
+
+   # Calculate the VWAP indicator in a reactive environment
+   vwapv <- shiny::reactive({
+     # Get model parameters from input argument
+     lookb <- input$lookb
+     # Calculate the VWAP indicator
+     closep <- closep()[, 1]
+     volum <- closep()[, 2]
+     vwapv <- HighFreq::roll_sum(tseries=closep*volum, lookb=lookb)
+     volumroll <- HighFreq::roll_sum(tseries=volum, lookb=lookb)
+     vwapv <- vwapv/volumroll
+     vwapv[is.na(vwapv)] <- 0
+     # Return the plot data
+     datav <- cbind(closep, vwapv)
+     colnames(datav) <- c(input$symbol, "VWAP")
+     datav
+   }) # end reactive code
+
+   # Return the dygraph plot to output argument
+   output$dyplot <- dygraphs::renderDygraph({
+     colv <- colnames(vwapv())
+     dygraphs::dygraph(vwapv(), main=paste(colv[1], "VWAP")) %>%
+     dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+     dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+     dySeries(name=colv[1], axis="y", label=colv[1], strokeWidth=2, col
+     dySeries(name=colv[2], axis="y2", label=colv[2], strokeWidth=2, col
+     }) # end output plot
+   }) # end server code
```

Reactive Event Handlers

Event handlers are functions which evaluate expressions when an event occurs (like a button press).

The functions `shiny::observeEvent()` and `shiny::eventReactive()` are event handlers.

The function `shiny::eventReactive()` returns a value, while `shiny::observeEvent()` produces a side-effect, without returning a value.

The function `shiny::reactiveValues()` creates a list for storing reactive values, which can be updated by event handlers.

```
> ## Define the server function
> servfun <- shiny::shinyServer(function(input, output) {
+
+   # Create an empty list of reactive values.
+   value_s <- reactiveValues()
+
+   # Get input parameters from the user interface.
+   nrows <- reactive({
+     # Add nrows to list of reactive values.
+     value_s*nrows <- input$nrows
+     input$nrows
+   }) # end reactive code
+
+   # Broadcast a message to the console when the button is pressed
+   observeEvent(eventExpr=input$button, handlerExpr={
+     cat("Input button pressed\n")
+   }) # end observeEvent
+
+   # Send the data when the button is pressed.
+   datav <- eventReactive(eventExpr=input$button, valueExpr={
+     # eventReactive() executes on input$button, but not on nrows()
+     cat("Sending", nrows(), "rows of data\n")
+     datav <- head(mtcars, input$nrows)
+     value_s$mpg <- mean(datav$mpg)
+     datav
+   }) # end eventReactive
+   #   datav
+
+   # Draw table of the data when the button is pressed.
+   observeEvent(eventExpr=input$button, handlerExpr={
+     datav <- datav()
+     cat("Received", value_s*nrows, "rows of data\n")
+     cat("Average mpg = ", value_s$mpg, "\n")
+     cat("Drawing table\n")
+     output$tablev <- renderTable(datav)
+   }) # end observeEvent
+
+ }) # end server code
>
```

Homework Assignment

Required

- Study all the lecture slides in [FRE7241_Lecture_2.pdf](#), and run all the code in [FRE7241_Lecture_2.R](#),

Recommended

- Download the files below from the [Share Drive](#)
- Read about *the risk parity strategy* from the files:
[Asness Leverage Risk Parity.pdf](#), [Clarke Risk Parity Minimum Variance Portfolios.pdf](#),
[Sorensen Risk Parity Portfolios.pdf](#).
- Read about *the seasonal anomalies* from the files:
[Matilde Sell in May Halloween Seasonal Anomaly.pdf](#), [Afik Sell in May Halloween Seasonal Anomaly.pdf](#), [Knuteson Overnight Seasonal Anomaly 2021.pdf](#), [Bogousslavsky Overnight Seasonal Anomaly.pdf](#), [Lachance Overnight Seasonal Anomaly.pdf](#).
- Read about *stop loss rules* from the file: [Bailey Prado Stop Loss Rules.pdf](#).
- Download all the shiny app files named `app_*.R` from the [Share Drive](#), and run them by pressing the "Run App" button in *RStudio*.
- Download all the *Rmarkdown* files named `*.Rmd` from the [Share Drive](#), and render them into *html* documents by pressing the "knit" button in *RStudio*.